COMUNICA+: Planejamento e desenvolvimento de um aplicativo móvel orientado a testes para a comunicação entre moradores e poder público

Luiz Henrique Garcia[1]

[1] Curso de Análise e Desenvolvimento de Sistemas – Universidade do Vale do Itajaí - Rua

{ luiz.6471030 }@edu.univali.br

Abstract. *This report presents the development of the Comunica+ mobile application, designed to facilitate communication between citizens and public administration through the registration, visualization, and prioritization of urban issues. The project was structured using agile methodologies, specifically the Scrum framework, and employed Test-Driven Development (TDD) practices to ensure functionality quality and reliability. The process encompassed planning, prototyping, development, testing, and validation, resulting in the creation of key features such as a problem registration form, an interactive map with multiple markers, and a feed with a voting system. Modern technologies—including React Native, Expo, TypeScript, Zustand, and Zod—were used to guarantee system robustness and scalability. Validation included functional testing, interface testing, and full user-flow verification, supported by documented evidence. The results demonstrate that Comunica+ meets the defined requirements and provides an efficient and accessible solution for enhancing community engagement in identifying urban issues.*

Keywords: Mobile development. React Native. Scrum. TDD. Urban application.

Resumo. *O presente relatório apresenta o desenvolvimento do aplicativo mobile Comunica+, concebido para facilitar a comunicação entre cidadãos e a gestão pública por meio do registro, visualização e priorização de problemas urbanos. O projeto foi estruturado com base em metodologias ágeis, especificamente o framework Scrum, e utilizou práticas de Test-Driven Development (TDD) para assegurar qualidade e confiabilidade das funcionalidades implementadas. O processo compreendeu planejamento, prototipação, desenvolvimento, testes e validação, com a criação de telas como registro de problemas, mapa interativo com múltiplos marcadores e feed de listagem com sistema de votos. Foram empregadas tecnologias modernas como React Native, Expo, TypeScript, Zustand e Zod para garantir robustez e extensibilidade ao sistema. A validação incluiu testes funcionais, de interface e verificação de fluxo completo, acompanhados de evidências documentais. Os resultados demonstram que o Comunica+ atende aos requisitos definidos, oferecendo uma solução eficiente e acessível para o engajamento da comunidade na identificação de demandas urbanas.*

Palavras-chave: *Desenvolvimento mobile. React Native. Scrum. TDD. Aplicativo urbano.*

1 Introdução

O desenvolvimento de soluções tecnológicas voltadas para dispositivos móveis tem se tornado cada vez mais relevante diante da necessidade de otimizar a comunicação entre cidadãos e administração pública. Problemas urbanos como buracos, falhas de iluminação, acúmulo de lixo e questões relacionadas à segurança exigem meios ágeis de registro, acompanhamento e priorização. Nesse contexto, o presente relatório apresenta o desenvolvimento do aplicativo Comunica+, produzido como parte do HANDS ON WORK VIII (HOW VIII), cujo objetivo é facilitar o envio e a visualização de ocorrências urbanas por meio de uma plataforma intuitiva e acessível.

A Etapa 1 do projeto concentrou-se no planejamento inicial, contemplando a geração de ideias, construção de mapas mentais, definição do escopo, identificação dos stakeholders,

levantamento das tarefas e organização das sprints utilizando metodologias ágeis. Além disso, foram realizadas atividades de concepção visual e modelagem inicial da solução, estabelecendo a base para o desenvolvimento do aplicativo.

Já a Etapa 2 correspondeu ao processo de implementação e validação do sistema. Nessa fase, foram desenvolvidas as principais funcionalidades do aplicativo, incluindo o registro de problemas, upload de imagens, classificação por categorias, exibição em feed, sistema de votos, integração com geolocalização e apresentação dos registros diretamente no mapa. Também foram aplicados testes seguindo o ciclo do Test-Driven Development (TDD), contemplando testes unitários e testes de fluxo, além da realização de ajustes e correções com base nas validações em dispositivos reais.

Assim, este relatório apresenta de forma estruturada todas as atividades realizadas ao longo do projeto, desde sua concepção inicial até a entrega de um MVP funcional, evidenciando a aplicação de boas práticas de Engenharia de Software, metodologias ágeis e técnicas de desenvolvimento mobile no contexto do HOW VIII.

2 Geração de Ideia

A geração de ideias foi o ponto de partida para o desenvolvimento do projeto *Comunica+*, um aplicativo mobile criado com o objetivo de fortalecer a comunicação entre os moradores e o poder público local. A proposta surgiu a partir da observação de problemas recorrentes enfrentados em diversos bairros, como buracos nas ruas, iluminação pública danificada, acúmulo de lixo e demora na resolução de demandas simples. Esses problemas, embora cotidianos, afetam diretamente a qualidade de vida da população e revelam uma dificuldade comum: a falta de um canal eficiente de diálogo entre a comunidade e a prefeitura.

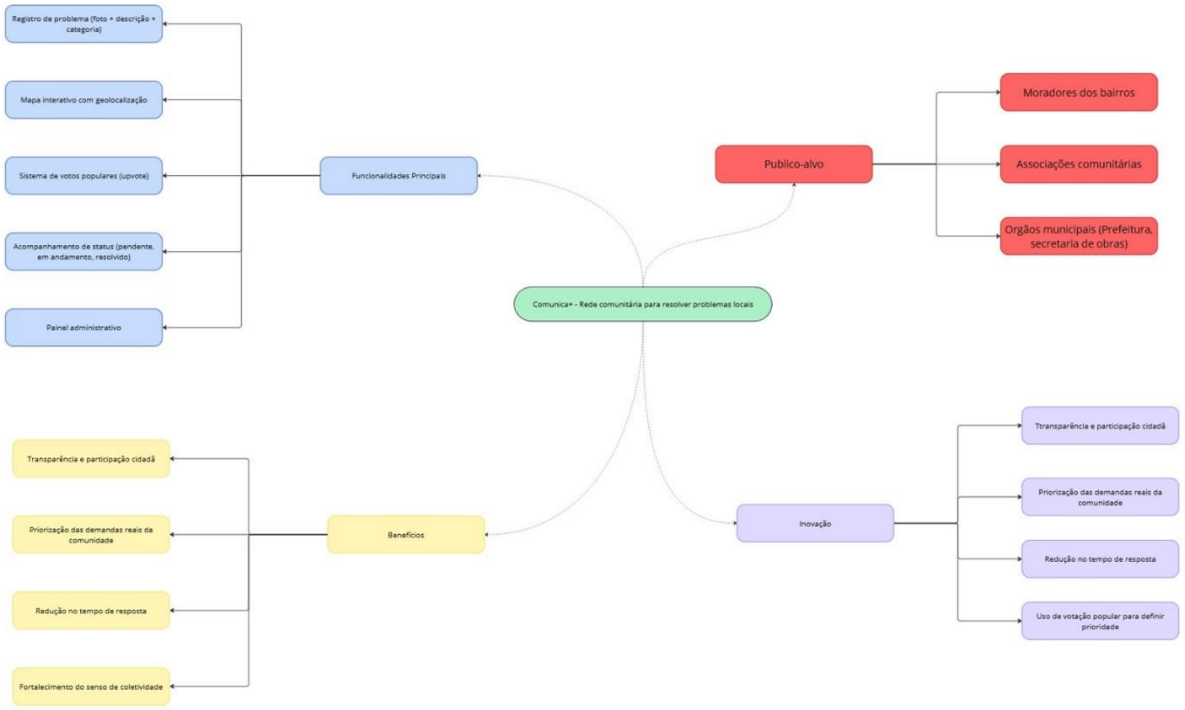
Durante o processo de ideação, foram aplicadas técnicas de brainstorming como na figura 1 e análise de necessidades locais. Inicialmente, foram levantadas diversas propostas, como um sistema de coleta seletiva, um aplicativo de segurança comunitária e uma plataforma de eventos locais. Entretanto, após análise de viabilidade e impacto social, a equipe optou pela criação de uma rede colaborativa de ouvidoria comunitária, que permitisse aos moradores registrarem problemas urbanos, votar nas demandas mais urgentes e acompanhar o status de cada solicitação.

Figura 1- Brainstorming

Fonte: autoria própria.

Para estimular a criatividade e definir a estrutura do projeto, também foram utilizados mapas mentais como na figura 2, que ajudaram a visualizar as funcionalidades principais e suas conexões. Entre as ideias discutidas, destacou-se o diferencial do *Comunica+*: um sistema de priorização por votação popular, no qual os próprios cidadãos determinam quais problemas devem ser resolvidos com mais urgência, promovendo uma forma democrática e transparente de gestão pública.

Figura 2 - Mapa Mental



Fonte: Autoria própria.

A partir dessa definição, iniciou-se a fase de refinamento da ideia, com foco em tornar o aplicativo simples, acessível e visualmente amigável. A proposta evoluiu para um modelo baseado em participação comunitária e validação contínua, onde o usuário não apenas reporta problemas, mas também participa ativamente da solução. Assim, o *Comunica+* se consolidou como uma ferramenta de engajamento social e cidadania digital, incentivando a colaboração entre moradores, líderes comunitários e o poder público.

3 Prova de Conceito

O projeto *Comunica+* teve como objetivo validar a viabilidade e relevância de uma plataforma mobile voltada para o registro, priorização e acompanhamento de problemas urbanos identificados pelos moradores de uma comunidade. Essa validação inicial foi realizada sem necessidade de protótipos complexos ou testes em ambientes reais, focando principalmente na clareza do conceito, na análise comparativa de soluções existentes e na obtenção de feedback construtivo.

A ideia que se busca validar é a criação de um aplicativo mobile colaborativo que permite aos cidadãos registrarem problemas urbanos (como buracos nas ruas, iluminação pública danificada, acúmulo de lixo entre outros), incluindo foto, descrição e categoria. Outros usuários

podem visualizar essas ocorrências e votar naquelas que consideram mais urgentes, gerando um ranking de prioridades baseado na opinião da própria comunidade. O diferencial do *Comunica+* está na adoção de um mecanismo de votação popular, o que fortalece a participação cidadã e torna as demandas mais transparentes, democráticas e orientadas pelas necessidades reais dos moradores.

Durante a análise, foram identificadas soluções semelhantes, como aplicativos de ouvidoria pública municipais, sistema Colab.re e outros como "Colab", "Cidadera", "Cidadão Online", "FixMyStreet" e "SeeClickFix". No entanto, esses serviços apresentam limitações: muitas vezes os usuários não conseguem acompanhar o status das demandas, não há participação comunitária direta na priorização e a comunicação costuma ser unidirecional. O *Comunica+* se diferencia por combinar registro público, geolocalização, votação entre moradores e transparência no status das solicitações, formando um fluxo participativo mais completo.

Na prática, o usuário abre o aplicativo, registra um problema urbano fazendo upload de uma foto, seleciona uma categoria e indica o local. A ocorrência aparece para todos os demais moradores, que podem votar caso considerem a situação urgente. Quanto mais votos, maior prioridade e visibilidade o problema ganha dentro da plataforma. Um painel administrativo, pensado para uso da prefeitura ou associações de bairro, pode alterar o status de cada ocorrência para "pendente", "em andamento" ou "resolvido", permitindo ao morador acompanhar a evolução publicamente. Esse modelo funciona como um "termômetro social", onde a opinião coletiva indica onde os recursos públicos devem ser aplicados primeiro.

A ideia foi apresentada a colegas, amigos e moradores. O feedback foi positivo, destacando principalmente:

(a) A falta de canais práticos para reportar problemas urbanos;
(b) Dificuldades em acompanhar o andamento das solicitações;
(c) A importância de envolver a comunidade na priorização.

As sugestões reforçaram o potencial social do aplicativo e indicaram pontos de atenção, como a necessidade de moderação para evitar publicações indevidas e a importância de manter uma interface simples.

Com base nos comentários recebidos, a ideia foi ajustada para incluir filtros de categoria, status de ocorrência e um painel administrativo simplificado que permita às prefeituras ou entidades comunitárias atualizar informações. Além disso, decidiu-se manter o foco exclusivamente em problemas urbanos, evitando que o aplicativo se torne genérico demais ou difícil de gerenciar.

Após a análise de soluções semelhantes, coleta de feedback e refinamentos, concluiu-se que o *Comunica+* apresenta um diferencial significativo ao unir transparência, engajamento social e priorização de demandas por votação popular. O modelo favorece o uso eficiente de recursos públicos, fortalece o senso de comunidade e promove cidadania digital.

4 Etapa 1 do projeto

4.1 Definição do Escopo do Projeto

O projeto *Comunica+* está alinhado ao tema "Planejamento e desenvolvimento de um projeto orientado a testes com programação mobile", aplicado à solução de problemas recorrentes na comunidade. Observou-se que moradores enfrentam dificuldades para reportar problemas urbanos, acompanhar o status de solicitações e priorizar demandas emergenciais. Dessa forma, o projeto visa desenvolver um aplicativo móvel colaborativo que permita o registro de problemas públicos com foto, categoria e localização, possibilitando que os próprios moradores votem nas ocorrências mais urgentes.

O desenvolvimento será conduzido seguindo uma abordagem orientada a testes (TDD – Test-Driven Development), garantindo que cada funcionalidade seja implementada de forma confiável desde o início, reduzindo erros e aumentando a qualidade do software. O projeto também será acompanhado por uma metodologia ágil baseada no framework Scrum, com planejamento, sprints e backlog configurados.

4.1.1 Compreensão do Problema na Comunidade

A comunidade apresenta dificuldades na comunicação com órgãos públicos sobre problemas urbanos, como buracos nas ruas, iluminação pública danificada, lixo acumulado e manutenção de praças. Além disso, muitos cidadãos não conseguem acompanhar o andamento de suas solicitações e desconhecem se outros moradores também enfrentam problemas semelhantes.

Para compreender melhor essas dores, foram realizadas conversas com moradores, observações de campo e análises em redes sociais comunitárias. Essas fontes reforçaram a necessidade de um canal digital simples, colaborativo e transparente, que permita reportar e acompanhar demandas de forma coletiva.

4.1.1.1 Exemplos de problemas

(a) Segurança (locais escuros);
(b) Mobilidade (buracos e calçadas danificadas);

(c) Saúde pública (lixo em excesso);
(d) Qualidade de vida;
(e) Acesso limitado a serviços públicos: dificuldade em registrar solicitações e atrasos na resposta;
(f) Falta de comunicação entre moradores: ausência de um canal centralizado que mostre demandas urbanas.

4.1.2 Definição Clara do Objetivo

O objetivo geral deste projeto é desenvolver um aplicativo móvel colaborativo que permita aos moradores registrarem problemas urbanos, votar nas ocorrências mais urgentes e acompanhar o andamento das soluções, a fim de promover transparência, participação cidadã e eficiência na priorização de demandas.

4.1.2.1 Exemplo de objetivo

(a) Facilitar o envio de reclamações urbanas com foto, descrição e geolocalização;
(b) Priorizar demandas por meio de votação comunitária;
(c) Permitir o acompanhamento do status de cada ocorrência (pendente, em andamento, resolvido);
(d) Fornecer painel administrativo para órgãos públicos ou associações de bairro;
(e) Assegurar qualidade e confiabilidade utilizando desenvolvimento orientado a testes (TDD);

4.1.3 Delimitação do Escopo

O escopo do projeto abrange funcionalidades essenciais para a operação e validação do *Comunica+*. Itens não relacionados ao problema principal serão excluídos para manter o foco e assegurar entregas dentro do prazo.

Incluído no Escopo:

(a) Registro de problemas urbanos com foto, descrição, categoria e localização;
(b) Votação popular para priorização de ocorrências;
(c) Exibição de lista e mapa de problemas registrados;
(d) Implementação de testes unitários, de integração e validação em dispositivos reais;

(e) Desenvolvimento mobile para Android com React Native + Expo;
(f) Entrega de um MVP funcional para testes com usuários.

Excluído do Escopo:

(a) Funcionalidades de rede social (comentários, chat, grupos);
(b) Gamificação avançada (conquistas, pontos distribuídos);
(c) Acompanhamento de status atualizado por administradores;
(d) Suporte para IOS;
(e) Testes manuais extensivos (o foco será em testes automatizados).

4.1.4 Identificação dos Stakeholders da Comunidade

Os principais stakeholders envolvidos são:

(a) Moradores da comunidade: usuários finais responsáveis por registrar e votar em ocorrências;
(b) Líderes comunitários e associações de bairro: fornecem apoio, divulgação e validação em campo;
(c) Órgãos públicos municipais: como secretaria de obras e zeladoria, responsáveis por agir sobre as demandas;
(d) Equipe de desenvolvimento: responsável pelo planejamento, implementação e testes do aplicativo;
(e) Testadores voluntários: fornecem feedback prático sobre usabilidade, performance e clareza do app.

4.1.5 Atividades e Tarefas Principais

A abordagem orientada a testes (TDD) organiza o ciclo do projeto em etapas repetitivas, garantindo alta qualidade e reduzindo retrabalho.

Principais atividades:

(a) Levantamento de Requisitos Funcionais: identificação das funcionalidades necessárias (registro, voto, status, mapa);
(b) Especificação de Testes: criação de testes unitários antes da implementação de cada função principal;

(c) Desenvolvimento das Funcionalidades: implementação do código que faz os testes passarem;
(d) Refatoração do Código: melhoria contínua da estrutura mantendo todos os testes aprovados;
(e) Testes de Integração: garantia de comunicação entre as telas e módulos (por exemplo, lista → detalhes → votos).

4.1.6 Ferramentas de Desenvolvimento e Teste

Para o desenvolvimento mobile e execução de testes serão utilizadas:

(a) React Native + Expo: framework multiplataforma para Android;
(b) VS Code: ambiente de desenvolvimento;
(c) Jest: testes unitários e de lógica;
(d) React Native Testing Library: testes de interface;
(e) Maestro: testes end-to-end mobile;
(f) Expo Go / Expo EAS Build: execução e geração de builds no dispositivo;
(g) GitHub: versionamento e integração contínua.

4.1.7 Métricas de Sucesso

Para avaliar a efetividade do projeto, serão observadas:

(a) Entrega do aplicativo mobile funcional;
(b) Cobertura de funcionalidades principais por testes automatizados;
(c) Redução de falhas e bugs durante o uso;
(d) Registro do ciclo TDD (testar → implementar → refatorar);
(e) Aplicativo executando com sucesso em pelo menos um smartphone Android;
(f) Capacidade de utilizar câmera, localização e interface sem falhas;
(g) Fluxo funcionando sem interrupções: Registrar problema → Votar → Visualizar status → Atualização do status;
(h) Coleta de feedback básico de usuários (colegas, moradores, professor);
(i) Percepção positiva sobre clareza, facilidade de uso e utilidade;
(j) Finalização dos sprints planejados dentro do cronograma;

4.1.8 Cronograma e Recursos

O cronograma de desenvolvimento do aplicativo Comunica+ foi estruturado utilizando a metodologia ágil Scrum, organizada em três sprints. Cada sprint possui duração definida,

conjunto de entregas específicas e foco de desenvolvimento. O acompanhamento é realizado pelo quadro do Trello, que registra tarefas, progresso e status de conclusão das atividades.

Sprint	**Duração**	**Foco**	**Principais Entregas**
Sprint 1	Semanas 1–2	Planejamento e Protótipo	Protótipo + Backlog configurado
Sprint 2	Semanas 3–4	Desenvolvimento e Testes	MVP funcional + testes TDD
Sprint 3	Semana 5	Validação e Entrega	App validado + vídeo + relatório final

Recursos Utilizados

Recursos Humanos

(a) Desenvolvedor: responsável pelo código, testes e documentação;
(b) Usuários voluntários: fornecem feedback durante testes;
(c) Professor/Cliente acadêmico: valida entregas e orientações.

Recursos Tecnológicos

(a) Linguagem/Framework: React Native + Expo (mobile);
(b) Sistema de testes: Jest, React Native Testing Library, Maestro;
(c) IDE/Editor: Visual Studio Code;
(d) Gerenciamento de tarefas: Trello (Scrum);
(e) Versionamento: GitHub;
(f) Emuladores: Expo Go, simuladores Android;
(g) Dispositivos reais: Smartphones Android.

Recursos Físicos

(a) Smartphones para teste;
(b) Internet estável;
(c) Computador para desenvolvimento.

Recursos de Software

(a) Expo SDK;
(b) Node.js;

(c) Bibliotecas de camera e geolocalização;
(d) Ícones e componentes de UI;
(e) Git para controle de versão.

5 Etapa 2

A Etapa 2 do projeto HOW VIII teve como foco o desenvolvimento prático, a implementação das funcionalidades previstas no planejamento inicial e a validação do aplicativo Comunica+ em ambiente real de uso. Após a fase de concepção apresentada na Etapa 1, esta etapa concentrou-se em transformar a solução proposta em um protótipo funcional, integrando elementos de interface, navegação, armazenamento de dados e recursos nativos do dispositivo, como câmera e geolocalização.

Durante esta fase, foram desenvolvido a interface do *Comunica+*, foi realizada uma prototipação inicial no Figma, permitindo visualizar a navegação, organização dos componentes e fluxo de interação do usuário antes da implementação. Essa etapa foi essencial para validar a usabilidade das telas, ajustar o posicionamento dos elementos e garantir que o design estivesse alinhado aos requisitos funcionais definidos no Product Backlog. A prototipação possibilitou maior clareza no desenvolvimento, reduzindo retrabalhos e servindo como referência visual durante toda a construção do aplicativo.

O fluxo completo de registro de ocorrências, o sistema de categorias, a exibição de problemas em formato de feed, o mapa interativo com marcadores personalizados e o mecanismo de votação. Além disso, a Etapa 2 incluiu a aplicação do ciclo do Test-Driven Development (TDD), envolvendo testes unitários e de comportamento, a análise de falhas, a correção de inconsistências e a verificação do aplicativo em um dispositivo móvel físico. Também foram executadas novas sprints utilizando a metodologia Scrum, permitindo organizar a implementação de forma incremental e iterativa.

Assim, esta etapa consolidou o MVP do Comunica+, resultando em um aplicativo funcional, testado e alinhado aos requisitos definidos anteriormente. A Etapa 2 representa, portanto, a transição do planejamento conceitual para a entrega prática da solução, demonstrando a integração entre teoria, metodologia e desenvolvimento mobile.

5.1 Aplicação, Testes e Validação de um Projeto Orientado a Testes com Programação Mobile

O desenvolvimento da aplicação Comunica+ na Etapa 2 foi conduzido com base em princípios de Engenharia de Software orientados a testes, seguindo o ciclo do Test-Driven Development (TDD). A abordagem permitiu que as funcionalidades fossem implementadas de forma incremental, garantindo maior confiança no código, previsibilidade no comportamento da aplicação e redução no surgimento de falhas durante o uso em dispositivos reais.

Inicialmente, foram estruturadas as principais telas e fluxos do aplicativo, incluindo registro de problemas, seleção de categorias, captura ou escolha de imagens, coleta automática de coordenadas por geolocalização, exibição dos registros em um feed ordenado e visualização dos problemas no mapa com pinos personalizados. O estado global da aplicação foi implementado com a biblioteca Zustand, garantindo organização, desacoplamento e assertividade na leitura e atualização dos dados pelo sistema.

No contexto de testes, TDD foi aplicado para validar funções críticas do aplicativo, tais como manipulação de dados, ordenação de registros, geração de identificadores, cálculo de votos e comportamento das funções de atualização. Foram realizados testes unitários utilizando Jest, assegurando que cada módulo funcionasse de forma independente e previsível. Além disso, foram realizados testes de fluxo e comportamento (end-to-end), verificando a interação completa do usuário com o sistema, como o processo de registrar um problema, visualizar no feed, navegar até o mapa e validar a exibição correta das informações no cartão do marcador.

A validação do aplicativo ocorreu tanto em ambiente de desenvolvimento quanto em dispositivo físico Android, permitindo identificar e corrigir inconsistências relacionadas ao layout, ao gerenciamento de estado, ao controle de campos obrigatórios e ao comportamento da câmera e da galeria. Funcionalidades como expansão de textos longos, exibição adaptativa de imagens e ajuste automático de marcadores no mapa foram continuamente testadas e refinadas, garantindo fluidez e usabilidade.

Com isso, a Etapa 2 consolidou a entrega de um MVP funcional, estável e devidamente validado, demonstrando a importância da integração entre testes, desenvolvimento e verificação contínua no contexto de programação mobile. A combinação entre TDD, testes unitários, validações práticas e ajustes iterativos assegurou que o Comunica+ atendesse às necessidades previstas no escopo e refletisse as boas práticas exigidas no HOW VIII.

5.1.1 Elaboração da Aplicação Mobile

A elaboração da aplicação mobile Comunica+ iniciou-se a partir da definição dos requisitos funcionais mínimos para construção do MVP (Minimum Viable Product). Nesta etapa, buscou-se transformar a proposta inicial do projeto em uma solução prática e operacional, capaz de registrar, armazenar e exibir ocorrências urbanas identificadas pelos usuários.

O processo de desenvolvimento foi estruturado com base em padrões modernos de programação mobile utilizando React Native aliado ao ecossistema Expo. Inicialmente, definiu-se a arquitetura geral da aplicação, contemplando a organização dos componentes, telas e estados globais. Para o gerenciamento de dados locais, adotou-se o Zustand, permitindo armazenar temporariamente as ocorrências no dispositivo do usuário, atendendo ao escopo da fase parcial do projeto.

A construção das telas principais constituiu a primeira etapa efetiva da elaboração. Foram desenvolvidos os seguintes módulos:

(a) Tela de Registro de Problemas, responsável pela entrada dos dados do usuário, incluindo título, categoria, descrição, bairro, cidade, foto e localização geográfica obtida via GPS;

(b) Tela de Feed, destinada à listagem das ocorrências registradas, exibindo as informações essenciais, foto, data, votos e estado atual;

(c) Tela de Mapa, responsável pela visualização espacial das ocorrências, utilizando marcadores customizados (pins) e permitindo interação direta com cada ponto no mapa;

Durante a implementação, estabeleceram-se diretrizes de experiência do usuário (UX) que priorizaram navegabilidade fluida, simplicidade visual e responsividade. Elementos como botões claros, cartões informativos, imagens redimensionáveis e hierarquia de texto foram projetados para facilitar o uso cotidiano.

A elaboração também envolveu a definição dos fluxos internos da aplicação, garantindo que os dados fossem validados corretamente antes do armazenamento. Implementaram-se mecanismos de prevenção de erros, como validação de campos obrigatórios, limitação de caracteres, manipulação adequada de imagens e atualização automática das coordenadas geográficas.

Por fim, a aplicação foi preparada para operar tanto no simulador quanto em dispositivos físicos Android, utilizando o recurso Expo Go durante o período de testes. Essa preparação garantiu que todos os componentes funcionassem conforme esperado, permitindo avançar para as etapas subsequentes de testes e validação.

5.1.1.1 Passos para Elaboração da Aplicação

A elaboração da aplicação mobile seguiu um conjunto de etapas sistemáticas baseadas em metodologias consolidadas de Engenharia de Software e desenvolvimento ágil. Os passos descritos a seguir foram fundamentados em autores e práticas amplamente aceitas no mercado, garantindo qualidade, rastreabilidade e coerência durante a construção do sistema.

5.1.1.1.1 Definição dos Requisitos

A primeira etapa consistiu na elicitação e definição dos requisitos funcionais e não funcionais da aplicação, conforme orientam Pressman (2016) e Sommerville (2019). Nesse momento, buscou-se compreender com clareza o problema real a ser resolvido: a necessidade de uma plataforma simples para registro e consulta de ocorrências urbanas pelos cidadãos.
Foram definidos requisitos essenciais, tais como:

(a) Registrar problemas urbanos com título, categoria, descrição, localização e imagem;
(b) Armazenar temporariamente as ocorrências no dispositivo do usuário;
(c) Exibir as ocorrências em um feed organizado e filtrável;
(d) Visualizar os problemas em um mapa interativo;
(e) Permitir que os usuários votem para destacar relevância.

Essa etapa garantiu que o escopo fosse alinhado ao MVP, evitando funcionalidades desnecessárias e mantendo o foco na solução principal.

5.1.1.1.2 Planejamento Arquitetural

Após a definição do escopo, realizou-se o planejamento arquitetural da aplicação, seguindo princípios de modularidade, simplicidade e separação de responsabilidades (Clean Architecture e boas práticas de projetos mobile).
A arquitetura definida contemplou:

(a) Organização em camadas, separando dados, lógica de negócio e interface;
(b) Gestão de estados com Zustand, evitando dependências pesadas e garantindo desempenho;
(c) Navegação estruturada utilizando React Navigation;
(d) Componentização das interfaces visando reutilização, clareza e facilidade de manutenção;

(e) Fluxo de dados unidirecional, evitando efeitos colaterais e tornando o comportamento da aplicação mais previsível.

Esse planejamento permitiu que o projeto tivesse robustez e escalabilidade, mesmo sendo um MVP.

5.1.1.1.3 Ferramentas e Ambiente de Desenvolvimento

Com base nas melhores recomendações do ecossistema JavaScript e do React Native, definiu-se um ambiente de desenvolvimento moderno e eficiente:

(a) Expo: facilitou o desenvolvimento, testes e acesso à câmera, GPS e armazenamento local;
(b) React Native: framework principal para construção de interfaces mobile multiplataforma;
(c) TypeScript: utilizado para maior segurança durante o desenvolvimento, evitando erros de tipagem;
(d) React Hook Form + Zod: empregados para validação de formulários, aumentando a confiabilidade dos dados;
(e) Visual Studio Code: como IDE padrão, com extensões de formatação e produtividade;
(f) Git e GitHub: para versionamento, backup e controle de evolução do projeto;
(g) EAS (Expo Application Services): preparação futura para builds, caso necessário.
(h) A escolha dessas ferramentas seguiu recomendações amplamente aceitas por comunidades profissionais e pela documentação oficial das tecnologias utilizadas.

5.1.1.1.4 Desenvolvimento Orientado a Testes (TDD)

Por fim, a abordagem de implementação seguiu princípios do Desenvolvimento Orientado a Testes (TDD), conforme proposto por Kent Beck (2003), em que o ciclo básico consiste em Red → Green → Refactor:

(a) Red: escrever um teste que falha;
(b) Green: implementar o código mínimo necessário para o teste passar;
(c) Refactor: melhorar o código garantindo que o teste continue passando.

Ainda que a aplicação seja um MVP, os princípios do TDD foram utilizados como guia para garantir:

(a) prevenção de erros frequentes;
(b) código mais limpo, coeso e com menor acoplamento;
(c) maior previsibilidade no comportamento dos componentes;
(d) validação constante durante o desenvolvimento.

Foram aplicados conceitos de validação imediata em formulários, tratamento de exceções, verificação de coordenadas geográficas e controle de estados como forma prática de incorporar os fundamentos do TDD ao projeto mobile.

5.1.2 Elaboração dos Testes

A etapa de elaboração dos testes do aplicativo Comunica+ foi conduzida com base em princípios estruturados da Engenharia de Software, com inspiração em Pressman (2016), Sommerville (2019) e nas práticas de Desenvolvimento Orientado a Testes (TDD) descritas por Kent Beck (2003). O objetivo foi garantir que cada funcionalidade implementada no aplicativo estivesse correta, estável, e adequadamente validada antes da integração com o restante do sistema.

Para isso, os testes foram planejados e executados a partir do fluxo real do usuário dentro do Comunica+, cobrindo todos os aspectos do aplicativo: registro de ocorrências, seleção de categorias, uso de GPS, captura de imagens, interação com o mapa, gerenciamento de estados, listagem no feed, visualização no mapa, rolagem automática, votos, validação com schemas Zod, e exibição dinâmica de detalhes.

A elaboração dos testes seguiu cinco pilares principais:

(a) **Testes Unitários**, que garantiram que funções isoladas e validações funcionassem de forma independente.
(b) **Testes de Integração**, que verificaram se componentes, módulos e estados funcionavam corretamente em conjunto.
(c) **Testes de Interface (UI Tests)**, que simularam a interação real do usuário, assegurando fluidez, navegação e comportamento visual adequado.
(d) **Testes Funcionais**, que validaram o funcionamento completo de fluxos reais, como registrar um problema e visualizá-lo no feed.

(e) **Testes de Regressão**, que garantiram que alterações posteriores — como melhorias no card dos pins e no feed — não quebrassem funcionalidades antigas.

Ao longo de todos os testes, buscou-se evitar erros comuns como inconsistência de coordenadas, falhas de layout, imagens deformadas, categorias incorretas, descrição com quebras inesperadas, estado desatualizado e falhas no mapeamento dos detalhes. Como o sistema é totalmente offline e armazenado localmente, testou-se também comportamento persistente em memória, reabertura do app e impacto de múltiplas ocorrências salvas.

5.1.2.1 Tipos de Testes

5.1.2.1.1 Testes Unitários

Os testes unitários foram planejados para validar partes pequenas e independentes do código. No Comunica+, isso incluiu principalmente:

(a) Validação dos dados com Zod (problemSchema)
(b) Cada campo do registro de problema possui uma validação específica:
(c) Título**:** mínimo de 5 caracteres.
(d) Descrição**:** entre 10 e 100 caracteres.
(e) Categoria**:** deve ser uma das categorias pré-definidas.
(f) Latitude e longitude**:** devem ser numéricas e válidas.
(g) Imagem**:** opcional, mas quando presente deve ser URI válida.

Exemplo de Teste Unitário:

(a) Inserir um título com apenas 3 caracteres → deve falhar.
(b) Inserir descrição com exatamente 10 caracteres → deve passar.
(c) Tentar salvar com latitude undefined → deve falhar.
(d) Categoria “Lixo” → deve passar.
(e) Categoria “ABC123” → deve falhar.

5.1.2.1.1.1.1 Testes sobre funções isoladas

Função normalize() no Feed: usada para busca sem acentos.
Ex.:

(a) Entrada: “Iluminação Pública”
(b) Saída esperada: “iluminacao publica”.

Função getPrettyAddress(): formata rua, bairro e cidade.

Testes unitários garantiram que endereços retornassem sempre o formato esperado mesmo com:

(a) bairro vazio,

(b) rua ausente,

(c) falha no reverseGeocode,

(d) valores nulos.

5.1.2.1.1.1.2 Testes de estados (Zustand)

Verificou-se:

(a) adicionar problema → incrementa array

(b) votar em problema → soma 1

(c) votar duas vezes → retorna false

(d) remover problema → exclui do array.

5.1.2.1.2 Testes de Integração

Os testes de integração verificaram se partes diferentes do app funcionavam corretamente juntas.

No Comunica+, isso foi fundamental para garantir o fluxo completo.

5.1.2.1.2.1 Integração formulário → estado global → feed

Fluxo testado:

(a) Usuário preenche todos os campos corretamente.

(b) Pressiona “Enviar”.

(c) Função addProblem salva o problema.

(d) FeedScreen deve exibir o item novo no topo.

Erros que foram testados e prevenidos:

(a) Problemas salvos com coordenadas incorretas.

(b) Feed não atualizar imediatamente após novo registro.

(c) Categorias exibidas incorretas no feed.

(d) Dados incompletos aparecerem no card.

5.1.2.1.2.2 Integração mapa → feed (focusId)

Testou-se:

(a) Pressionar um pin no mapa → ir para o feed.
(b) Feed rolar automaticamente até o item correto.
(c) Expandir automaticamente caso o usuário esteja vindo do mapa.

Trechos testados:

(a) scroll automático;
(b) expandedId correto;
(c) limpeza do focusId após o uso.

5.1.2.1.2.3 c) Integração imagem → card no feed → card no mapa

A imagem precisava:

(a) manter o aspectRatio,
(b) não deformar,
(c) não estourar limite de 260px,
(d) funcionar com diferentes orientações (horizontal, vertical).

Testou-se:

(a) foto vertical 1080x1920
(b) foto horizontal 1920x1080
(c) imagem quadrada 800x800
(d) ausência de imagem (fallback).

5.1.2.1.2.4 Integração entre categorias e UI

Após adicionar categorias no registrar problema, verificou-se:

(a) categoria refletindo no card do feed;
(b) categoria refletindo no pin do mapa;
(c) destaque visual em “Aberto”.

5.1.2.1.3 Testes de Interface de Usuário (UI Tests)

Os testes de UI verificaram o comportamento visual e interativo da aplicação.
Testes realizados:

5.1.2.1.3.1 Teste do card de problema no feed

Foram verificados:

(a) tamanho correto da imagem;
(b) “Ver mais / Ver menos” funcionando para textos grandes;
(c) textos pequenos não exibirem o botão;
(d) bordas, espaçamentos, cores e tipografia;
(e) destaque da categoria (“Aberto” em amarelo).

5.1.2.1.3.2 Teste no Mapa

(a) Pin exibido corretamente (PNG com tamanho ajustado).
(b) Card flutuante exibindo apenas título, categoria e parte da descrição.
(c) Card expandido com rolagem se texto longo.
(d) Botão “Ver detalhes” funcionando.

5.1.2.1.3.3 Dropdown de categorias no cadastro

(a) Abrir e fechar corretamente.
(b) Seleção aplicando o valor automaticamente.
(c) Fechar ao clicar em um item.

5.1.2.1.3.4 Campo de descrição com altura dinâmica

(a) Crescimento suave até 260px.
(b) Caracteres contando corretamente.
(c) Layout não quebrando.

5.1.2.1.3.5 Busca no feed

(a) Abrir e fechar campo de busca.

(b) Resetar estado quando usuário usa mapa → feed.
(c) Normalização de acentos funcionando.

5.1.2.1.4 Testes Funcionais

Os testes funcionais validaram o comportamento completo do aplicativo.
Foram testados fluxos reais como:

5.1.2.1.4.1 Fluxo 1 – Registrar um problema

(a) Preencher título, categoria, cidade, bairro e descrição.
(b) Marcar localização no mapa.
(c) Tirar foto ou escolher da galeria.
(d) Pressionar "Enviar".
(e) Problema aparecer imediatamente no feed.

Testes verificaram:

(a) validações Zod funcionando;
(b) campos obrigatórios;
(c) comportamento do mapa;
(d) preview da imagem;
(e) limite de 100 caracteres na descrição.

5.1.2.1.4.2 Fluxo 2 – Buscar problema no feed

(a) buscar por título ("Buraco");
(b) buscar por categoria ("Iluminação");
(c) buscar por nome da rua (via reverseGeocode);
(d) buscar acento e sem acento ("Iluminação" / "Iluminacao").

5.1.2.1.4.3 Fluxo 3 – Ver no mapa

(a) clicar em "Mapa" no card → abrir a tela do mapa;
(b) aplicar o focusId;
(c) centralizar o mapa;

(d) destacar pin selecionado.

5.1.2.1.4.4 Fluxo 4 – Votar

(a) votar 1 vez → +1;
(b) tentar votar de novo → erro “Você já votou neste problema”;
(c) feed atualizar contagem;
(d) votos persistidos no Zustand local.

5.1.2.1.5 Testes de Regressão

Sempre que uma nova alteração foi feita no app, testes de regressão garantiram que nada quebrasse.
Situações em que regressões foram executadas:

Ajuste do card do mapa
Ao adicionar categoria, endereço e botão “Ver detalhes”, verificou-se:

(a) imagens continuavam funcionando;
(b) rolagem automática do feed não quebrou;
(c) showMore não travou;
(d) texto longo não empurrava layout para fora.

Ajuste do card do feed
Quando incluímos:

(a) destaque da categoria;
(b) lógica nova para texto com número de linhas;
(c) iconografia atualizada;

Testou-se:

(a) feed não quebrava no scroll;
(b) rolagem automática do mapa → feed continuava funcionando.

Ajustes nos pins
Testou-se:

(a) PNG com tamanho alterado;

(b) não deformar em diferentes resoluções;
(c) card do mapa não deslocando o pin.

Formulário do registro

Ao implementar altura dinâmica do campo de descrição, verificou-se:

(a) dropdown de categorias continuava funcionando;
(b) preview de imagem não era afetado;
(c) mapa continuava recebendo coordenadas corretamente.

5.1.2.2 Ferramentas para Testes Mobile

A elaboração, execução e validação dos testes do aplicativo Comunica+ foram apoiadas por um conjunto de ferramentas integradas ao ambiente de desenvolvimento mobile. Essas ferramentas permitiram tanto a automação de partes do processo de verificação, quanto a realização de testes manuais exploratórios em dispositivos reais e emuladores, garantindo maior confiabilidade ao produto final. A seguir, são descritas as principais ferramentas utilizadas no projeto.

5.1.2.2.1 Expo e Expo Go

O Expo foi a base do ambiente de desenvolvimento do Comunica+, permitindo compilar, rodar e testar rapidamente o aplicativo em modo de desenvolvimento. Em conjunto com o Expo Go, instalado em um dispositivo Android físico, foi possível executar o aplicativo em tempo real, testar fluxos completos e validar o comportamento das funcionalidades diretamente no celular. Por meio do Expo Go, foram testados:

(a) o fluxo de registro de problemas (preenchimento de formulário, validação de campos e envio);
(b) o comportamento da câmera e da galeria ao capturar ou selecionar imagens;
(c) a obtenção da localização atual do usuário via GPS;
(d) a exibição dos problemas no feed e no mapa;
(e) o funcionamento dos botões de ação, como “Usar minha localização”, “Mapa”, “Votar” e “Ver no feed”.

Essa combinação permitiu uma validação contínua do Comunica+ em ambiente real, com recarregamento rápido (hot reload) e observação imediata dos efeitos de cada alteração de código.

5.1.2.2.2 Emulador Android e Dispositivos Físicos

Além do uso do Expo Go, foram utilizados **emuladores Android** e **dispositivos físicos** para testar o aplicativo em diferentes resoluções e condições de uso. Essa etapa foi importante para:

(a) verificar a responsividade dos componentes de interface (cards, lista, mapa, botões);
(b) garantir que imagens verticais e horizontais fossem exibidas corretamente (sem deformações ou cortes inadequados);
(c) observar o comportamento do mapa com diferentes níveis de zoom e posições geográficas;
(d) validar o desempenho da aplicação com múltiplos registros de problemas salvos.

Os testes em dispositivo físico também permitiram validar recursos dependentes de hardware, como GPS e câmera, em uma situação mais próxima do uso real.

5.1.2.2.3 Jest (Testes Unitários)

Para a camada de testes automatizados, foi adotado o Jest como ferramenta principal de testes unitários. O Jest é um framework amplamente utilizado no ecossistema JavaScript/TypeScript e foi empregado para validar partes específicas da lógica do Comunica+, como:

(a) funções auxiliares de normalização de texto (por exemplo, remoção de acentos na busca do feed);
(b) formatação de endereços, combinando rua, bairro e cidade;
(c) comportamento de funções relacionadas ao estado global (como incremento de votos, adição de novos problemas e prevenção de votos duplicados).

Esses testes ajudaram a garantir que a lógica de negócio permanecesse correta mesmo após ajustes e refatorações, reduzindo o risco de regressões silenciosas.

5.1.2.2.4 Zod e React Hook Form (Validação como teste de entrada)

Embora não sejam ferramentas de teste no sentido tradicional, o Zod e o React Hook Form desempenharam um papel fundamental na validação dos dados e, consequentemente, na qualidade geral do aplicativo.

O Zod foi utilizado para definir o problemSchema, responsável por validar:

(a) tamanho mínimo de título e descrição;
(b) obrigatoriedade de cidade, bairro e descrição;
(c) consistência de latitude e longitude;
(d) tipos dos campos (string, número, etc.).

Esse schema atua como uma "camada de teste" na entrada de dados, impedindo que registros inválidos sejam aceitos, o que reduz a necessidade de tratamentos posteriores.

O React Hook Form, por sua vez, foi responsável por integrar essa validação à interface, exibindo mensagens de erro, controlando estados de formulário e evitando o envio de informações incompletas. Dessa forma, erros comuns de uso foram identificados e tratados ainda na camada de interface.

5.1.2.2.5 Metro Bundler, Logs e Console

O Metro Bundler, em conjunto com o uso de console.log e mensagens de erro exibidas no terminal, foi uma ferramenta essencial para o processo de testes e depuração. Durante o desenvolvimento, foram utilizados logs para:

(a) inspecionar o conteúdo dos objetos de problema salvos no Zustand;
(b) verificar se as coordenadas estavam corretas ao arrastar o marcador no mapa;
(c) conferir a formatação de endereços obtidos via reverseGeocodeAsync;
(d) analisar o comportamento dos estados expanded, focusId e selected nas telas de feed e mapa.

Esses recursos permitiram identificar rapidamente falhas lógicas, comportamentos inesperados e efeitos colaterais, facilitando a correção ainda na fase de desenvolvimento.

5.1.2.2.6 Visual Studio Code, Extensões e GitHub

O Visual Studio Code (VS Code) foi a IDE utilizada para escrever o código do Comunica+. Embora não seja uma ferramenta de teste em si, contribuiu diretamente para a qualidade do código e, consequentemente, para a qualidade dos testes, por meio de:

(a) destaque de sintaxe e autocompletar;
(b) extensões de formatação e linting (como ESLint/Prettier, quando configurados);
(c) integração com o terminal, facilitando a execução de comandos de testes e builds;
(d) integração com o Git e o GitHub, possibilitando versionamento, registro de mudanças e recuperação de versões estáveis do projeto.

O uso do GitHub também possibilitou testar em diferentes máquinas, guardar versões importantes do aplicativo e garantir um histórico de evolução do código, o que impacta diretamente na capacidade de reproduzir e validar comportamentos.

Em conjunto, essas ferramentas formaram um ecossistema completo de desenvolvimento e testes para o Comunica+, permitindo desde a escrita do código, sua validação automática e manual, até a execução em ambiente real de uso. Isso garantiu que a aplicação fosse construída e verificada segundo boas práticas de programação mobile e de garantia de qualidade de software.

5.1.3 Validação do Projeto

A validação do projeto Comunica+ teve como objetivo verificar se o aplicativo, em sua forma de MVP, atende aos requisitos definidos na etapa inicial do HOW VIII e se o comportamento observado durante o uso é consistente, estável e adequado ao cenário de aplicação proposto. Esse processo envolveu tanto a avaliação técnica das funcionalidades implementadas quanto a análise prática do fluxo de uso em dispositivos reais, buscando garantir que a solução seja aplicável ao contexto de registro de problemas urbanos por cidadãos.

Inicialmente, foi realizada uma validação de requisitos, na qual o aplicativo foi confrontado com o escopo definido na fase de concepção. Confirmou-se que o Comunica+ permite o registro de ocorrências urbanas contendo título, categoria, cidade, bairro, descrição, localização geográfica e imagem opcional. Também foi validado que as ocorrências são exibidas em um feed organizado, com destaque para informações essenciais, como votos, data e status. Além

disso, confirmou-se a presença do mapa interativo, que exibe os problemas com marcadores personalizados (pins) e permite ao usuário visualizar detalhes ao selecionar um ponto no mapa.
A validação funcional incluiu a execução de fluxos completos de uso. Testou-se o processo de registrar um problema passando por todas as etapas: preenchimento dos campos obrigatórios, uso da câmera ou da galeria para anexar uma imagem, seleção ou ajuste da localização no mapa e, por fim, envio do registro. Em seguida, verificou-se que o problema aparece corretamente no feed, com título, categoria, endereço formatado, contagem de votos, texto da descrição e imagem ajustada com o aspecto correto. Validou-se também a navegação do feed para o mapa por meio do botão "Mapa", bem como o comportamento inverso, em que, a partir do mapa, é possível focar um problema específico e, se necessário, visualizar seus detalhes no feed.
Outro ponto importante da validação consistiu na experiência de uso e interface. O layout foi testado em diferentes tamanhos de tela e orientações de imagem (vertical, horizontal e quadrada), garantindo que os cartões de problema mantivessem legibilidade, espaçamento adequado e hierarquia visual clara. Foram validados elementos como o botão "Ver mais/Ver menos" em descrições longas, evitando que textos extensos quebrassem o layout, bem como o comportamento do campo de descrição de altura dinâmica no formulário de registro. Também se verificou o funcionamento do destaque visual para o status "Aberto", tanto no card do mapa quanto no feed, colaborando para a rápida identificação da situação da ocorrência.
A validação técnica e de comportamento de dados abrangeu o uso do estado global através do Zustand, assegurando que a adição de novos problemas, a atualização de votos e a leitura dos dados em diferentes telas ocorressem de forma consistente. Testes práticos confirmaram que o usuário não consegue votar duas vezes no mesmo problema e que a contagem de votos é atualizada corretamente no feed. Além disso, a validação das coordenadas mostrou que a localização exibida no mapa corresponde às informações presentes no registro, e que a função de uso da localização atual posiciona corretamente o marcador inicial do mapa.
Do ponto de vista de confiabilidade dos dados, a validação com o schema de validação (Zod) e o React Hook Form mostrou-se eficaz no bloqueio de registros inválidos, como títulos muito curtos, descrições incompletas ou ausência de campos obrigatórios. Durante os testes, foram intencionalmente inseridos dados incorretos para verificar a reação da aplicação, confirmando o aparecimento de mensagens de erro e a prevenção do envio de formulários incompletos.
A validação em dispositivo físico Android, utilizando o Expo Go, permitiu observar o comportamento do aplicativo em condições próximas ao uso real. Nessa fase, foram analisados o desempenho geral, o tempo de resposta, a fluidez da navegação entre telas, o tempo de carregamento do mapa e a integração com a câmera e a galeria. Pequenos ajustes de layout e

espaçamento foram realizados a partir dessa etapa, aprimorando a experiência do usuário e reduzindo pontos de atrito na interação.

Por fim, a validação do projeto confirmou que o Comunica+ atende de forma satisfatória ao objetivo proposto no HOW VIII: oferecer uma aplicação mobile simples, funcional e confiável para registro e visualização de problemas urbanos. Embora o aplicativo ainda possa ser ampliado com novas funcionalidades, como autenticação avançada, sincronização com serviços externos ou integração com APIs municipais, o MVP entregue demonstrou robustez, coerência com o escopo definido e aderência às boas práticas de desenvolvimento mobile e de testes adotadas ao longo do projeto.

5.1.3.1 Passos para a Validação

A validação do projeto Comunica+ foi organizada em etapas sequenciais, de forma a garantir que cada aspecto da aplicação — requisitos, fluxos, interface, dados e desempenho — fosse verificado de maneira sistemática. A seguir, são apresentados os principais passos adotados para validar o aplicativo em sua versão de MVP.

5.1.3.1.1 Definição dos critérios de validação

O primeiro passo consistiu em definir critérios claros de validação, com base no escopo descrito na Etapa 1 do HOW VIII. Foram estabelecidos como critérios principais:

(a) permitir o registro completo de um problema urbano (título, categoria, cidade, bairro, descrição, localização e imagem);
(b) exibir as ocorrências em um feed organizado, com informações essenciais e possibilidade de interação;
(c) apresentar os problemas em um mapa interativo, com marcadores (pins) que permitissem identificar rapidamente o local e o resumo do problema;
(d) possibilitar a votação em problemas, com controle para evitar votos duplicados;
(e) garantir que o aplicativo funcionasse de forma estável em dispositivo físico Android.

Esses critérios serviram como referência para todas as demais etapas da validação.

5.1.3.1.2 Validação dos requisitos implementados

Em seguida, foi realizada uma validação de requisitos, comparando o que havia sido implementado com o que havia sido planejado inicialmente. Nessa etapa, foram conferidos:

(a) se o formulário de registro de problemas continha todos os campos previstos;
(b) se a validação de dados (via Zod e React Hook Form) respeitava os limites mínimos de caracteres e obrigatoriedade dos campos;
(c) se as categorias cadastradas eram as mesmas definidas no planejamento;
(d) se o mapa utilizava as coordenadas corretas e exibia o marcador na posição esperada;
(e) se a tela de feed listava corretamente os problemas, trazendo título, categoria, endereço, data, votos e status.

Sempre que algum requisito não era atendido plenamente, o código era ajustado e o fluxo voltava a ser revalidado.

5.1.3.1.3 Validação dos fluxos principais do usuário

Depois disso, foram testados os fluxos principais de uso, simulando o comportamento real de um usuário do Comunica+:

(a) Fluxo de registro: preencher o formulário, marcar o local no mapa, tirar ou escolher uma foto e enviar o problema.
(b) Fluxo de consulta no feed: localizar o problema recém-criado, verificar suas informações, conferir a imagem e o texto completo, usar o “Ver mais/Ver menos” em descrições longas.
(c) Fluxo de consulta no mapa: acessar o mapa, visualizar o marcador correspondente, abrir o card do problema e checar título, categoria, endereço e descrição.
(d) Fluxo de navegação entre telas: ir do feed para o mapa e do mapa para o feed utilizando os botões e o focusId, garantindo que o problema correto fosse focalizado.

Esse passo permitiu validar não apenas as funcionalidades isoladas, mas também a coerência do sistema como um todo.

5.1.3.1.4 Validação da interface e experiência do usuário (UX)

A validação também envolveu uma análise cuidadosa da interface gráfica e da experiência do usuário. Foram observados:

(a) o comportamento do card no feed, incluindo alinhamento de textos, tamanhos de fonte, espaçamentos e cores;
(b) o ajuste automático da altura do campo de descrição no formulário, garantindo conforto na digitação sem quebrar o layout;
(c) a exibição correta de imagens em formatos verticais, horizontais e quadrados, evitando distorções e cortes indesejados;
(d) o funcionamento do botão "Ver mais/Ver menos" nas descrições longas, garantindo que textos grandes não comprometessem a legibilidade;
(e) o destaque visual do status "Aberto" e da categoria nas telas de feed e mapa, para rápida identificação.

Correções de layout, espaçamento e hierarquia visual foram feitas com base nessa etapa de validação.

5.1.3.1.5 Validação do comportamento dos dados e do estado global

Outro passo importante foi a validação do comportamento dos dados e do estado global controlado pelo Zustand. Foram validados:

(a) a adição de um novo problema ao estado;
(b) a atualização da contagem de votos de forma consistente no feed;
(c) a prevenção de votos duplicados por um mesmo usuário;
(d) a correta associação entre o problema salvo e as coordenadas exibidas no mapa;
(e) a utilização correta do focusId ao navegar entre mapa e feed, garantindo que o item certo fosse focalizado e, se necessário, expandido.

Esse passo assegurou que os dados do aplicativo permanecessem coerentes durante toda a navegação.

5.1.3.1.6 Validação em emulador e dispositivo físico

Após a validação em ambiente de desenvolvimento, o aplicativo foi testado tanto em emuladores Android quanto em dispositivo físico, utilizando o Expo Go. Nessa etapa, foram verificados:

(a) o desempenho geral da aplicação, incluindo tempo de abertura e navegação entre telas;
(b) o carregamento do mapa e o tempo de renderização dos pinos;
(c) o funcionamento da câmera e da galeria, especialmente em redes mais lentas;
(d) o comportamento da geolocalização ao utilizar a opção "Usar minha localização";
(e) a estabilidade do app com múltiplos registros de problemas cadastrados.

Esses testes permitiram identificar ajustes finos necessários para melhorar a experiência real de uso.

5.1.3.1.7 Revalidação após correções (feedback e regressão)

Sempre que uma falha, inconsistência ou problema visual era identificado durante os testes, o código era corrigido e o fluxo correspondente voltava a ser revalidado. Em muitos casos, isso envolveu testes de regressão, como:

(a) reabrir o formulário para conferir se a validação ainda funcionava após ajustes;
(b) criar novos problemas para garantir que o feed continuasse exibindo-os corretamente;
(c) testar novamente o foco no mapa e a expansão de cards após alterações na interface;
(d) conferir se imagens e descrições continuavam corretas depois de mudanças em estilos e componentes.

Esse ciclo de correção e revalidação foi repetido até que o comportamento do aplicativo se mostrasse estável e alinhado aos critérios definidos no início da validação.

5.1.4 Ciclo de Desenvolvimento e Validação com TDD

O desenvolvimento do aplicativo Comunica+ foi guiado por princípios do Desenvolvimento Orientado a Testes (TDD – Test-Driven Development), ainda que de forma adaptada à realidade

de um projeto acadêmico e de um MVP mobile. O TDD tem como objetivo central garantir que o código seja escrito de maneira incremental, sempre baseado em testes e na verificação constante do comportamento esperado do sistema. No contexto do Comunica+, esse ciclo foi aplicado principalmente na validação das regras de negócio, na consistência dos dados, nos fluxos principais de uso e na prevenção de regressões a cada ajuste de layout ou funcionalidade. De forma geral, o TDD segue um ciclo iterativo composto pelas etapas escrever o teste, ver o teste falhar, implementar o código mínimo necessário, refatorar o código e garantir que não haja regressões. No Comunica+, esse ciclo se expressou tanto em testes automatizados (usando Jest e validações com Zod) quanto em testes práticos realizados diretamente no emulador e em dispositivo físico, sempre com o objetivo de reduzir erros, aumentar a previsibilidade do comportamento da aplicação e assegurar a qualidade do produto final.
A seguir, são descritos os passos do ciclo de desenvolvimento e validação com TDD, contextualizados ao projeto Comunica+:

5.1.4.1 Escrever um teste

O primeiro passo consiste em definir, de forma clara, o comportamento esperado de uma funcionalidade antes da sua implementação completa. No Comunica+, isso significou, por exemplo:

(a) especificar como o **schema de validação** (problemSchema com Zod) deveria se comportar ao receber um título curto demais, uma descrição incompleta ou coordenadas inválidas;
(b) definir que a função de **normalização de texto** usada na busca do feed deveria remover acentos e transformar tudo em minúsculas;
(c) estabelecer que a função responsável por **incrementar votos** deveria impedir votos duplicados do mesmo usuário.

Mesmo quando o teste não era inicialmente automatizado, o comportamento esperado era escrito, pensado e descrito antes da implementação, o que ajudou a orientar a forma como o código seria desenvolvido.

5.1.4.2 Fazer o teste falhar

Em seguida, o teste é executado sobre uma funcionalidade ainda inexistente ou incompleta, o que naturalmente resulta em falha. No Comunica+, isso ocorreu, por exemplo, quando:

(a) o schema de Zod era testado com valores inválidos (título de 3 caracteres, descrição muito curta, ausência de cidade ou bairro) e o sistema inicialmente não tratava todas as situações;

(b) ao simular votos repetidos, o código ainda permitia incremento indevido da contagem;

(c) ao testar a busca no feed com termos acentuados e sem acento, verificou-se que os resultados não eram retornados corretamente.

Essa etapa é importante porque confirma que o teste é **válido** (ou seja, detecta o problema) e que a funcionalidade ainda não está pronta, evitando uma falsa sensação de segurança.

5.1.4.3 Escrever o código

Após observar a falha, passa-se à implementação do código necessário para que o teste seja aprovado. No Comunica+, isso envolveu:

(a) ajustar o problemSchema para exigir comprimento mínimo de título e descrição, bem como obrigatoriedade de cidade e bairro;

(b) implementar lógica de bloqueio de **voto duplicado**, vinculando o voto ao usuário e retornando uma indicação de erro quando uma segunda tentativa era detectada;

(c) aprimorar a função de normalização de texto da busca, utilizando técnicas como remoção de acentos (normalize("NFD")) e transformação para minúsculas, garantindo que “Iluminação” e “iluminacao” fossem tratados da mesma forma;

(d) ajustar o cálculo e a formatação de endereço a partir do reverseGeocodeAsync, garantindo que rua, bairro e cidade fossem corretamente concatenados.

Essa etapa tem foco em fazer o teste passar, ainda que com o código mais simples possível para atingir o comportamento desejado.

5.1.4.4 Refatorar

Com os testes passando, entra-se na fase de refatoração, em que o código é melhorado sem alteração do seu comportamento externo. No Comunica+, a refatoração foi aplicada, por exemplo, ao:

(a) extrair funções auxiliares para evitar duplicação de lógica (como formatação de endereço e normalização de texto);
(b) reorganizar componentes de interface, reaproveitando estilos e simplificando a estrutura do JSX;
(c) limpar trechos de código que ficavam redundantes após introdução de novas abstrações, como a centralização do cálculo de aspectRatio de imagens ou o tratamento de estados em telas como Feed e Mapa.

Durante essa etapa, o conjunto de testes (automatizados e práticos) é executado novamente para garantir que as melhorias internas não alteraram o comportamento esperado.

5.1.4.5 Executar testes de regressão

Por fim, toda vez que uma nova funcionalidade foi adicionada ou algum ajuste significativo foi realizado, foram executados testes de regressão para garantir que funcionalidades já existentes não fossem quebradas. No contexto do Comunica+, isso ocorreu em várias situações, tais como:

(a) após modificar o card do mapa para exibir **categoria, endereço e botão "Ver mais/Ver menos"**, foi necessário testar novamente o funcionamento do foco no mapa, a exibição dos pins e a transição para o feed;
(b) após ajustar o comportamento de descrições longas no feed e no mapa, revalidou-se o layout para garantir que textos menores continuassem sendo exibidos corretamente e que o botão "Ver mais" só aparecesse quando realmente necessário;
(c) depois de melhorar a altura dinâmica do campo de descrição no formulário de registro, foram refeitos os fluxos de cadastro para assegurar que a validação não havia sido comprometida;
(d) após qualquer alteração em estados globais (como votos, lista de problemas e foco em itens), foram retestados todos os fluxos principais para confirmar a consistência do comportamento entre as telas.

Esse ciclo contínuo de Testar → Desenvolver → Refatorar → Revalidar permitiu que o Comunica+ evoluísse de forma controlada, minimizando o surgimento de erros, aumentando a confiança no código e garantindo que as melhorias visuais e funcionais fossem incorporadas sem comprometer a estabilidade do aplicativo.

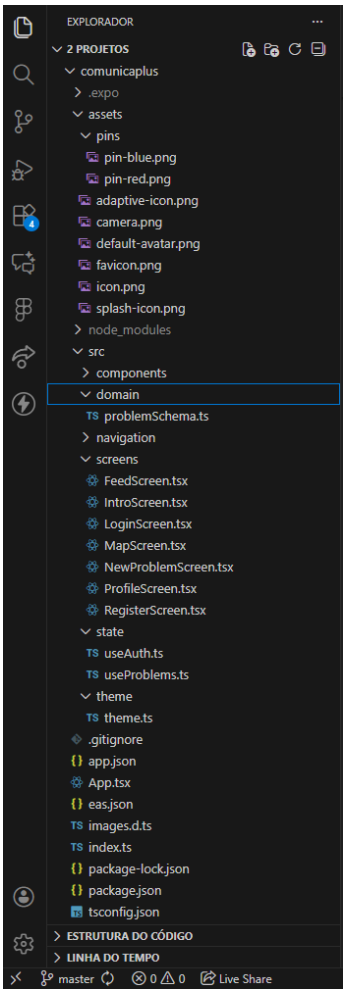
5.1.5 Evidências do Desenvolvimento, Testes e Validação

A presente seção reúne todas as evidências visuais que comprovam o processo de desenvolvimento, implementação, testes e validação do aplicativo Comunica+, contemplando desde a estrutura inicial do projeto até o funcionamento final das telas em dispositivo físico. As imagens apresentadas demonstram as etapas práticas realizadas, incluindo elaboração da interface, processamento dos dados, execução das funcionalidades principais, correção de falhas identificadas durante os testes e validação dos fluxos do usuário. Todas as figuras abaixo foram produzidas pelo autor durante o desenvolvimento do projeto, registrando fielmente o comportamento da aplicação em ambiente real.

5.1.5.1 Evidência do PROJETO e CÓDIGO (VSCode)

A Figura 3 apresenta a estrutura de diretórios do projeto *Comunica+* no ambiente de desenvolvimento Visual Studio Code. Nela, é possível observar a organização modular do sistema, dividida em pastas como screens, state, domain, theme e assets, que agrupam os componentes de interface, gerenciamento de estado, validações, estilos e recursos gráficos da aplicação. Essa estruturação garante maior clareza, separação de responsabilidades e facilidade de manutenção durante o desenvolvimento do aplicativo mobile.

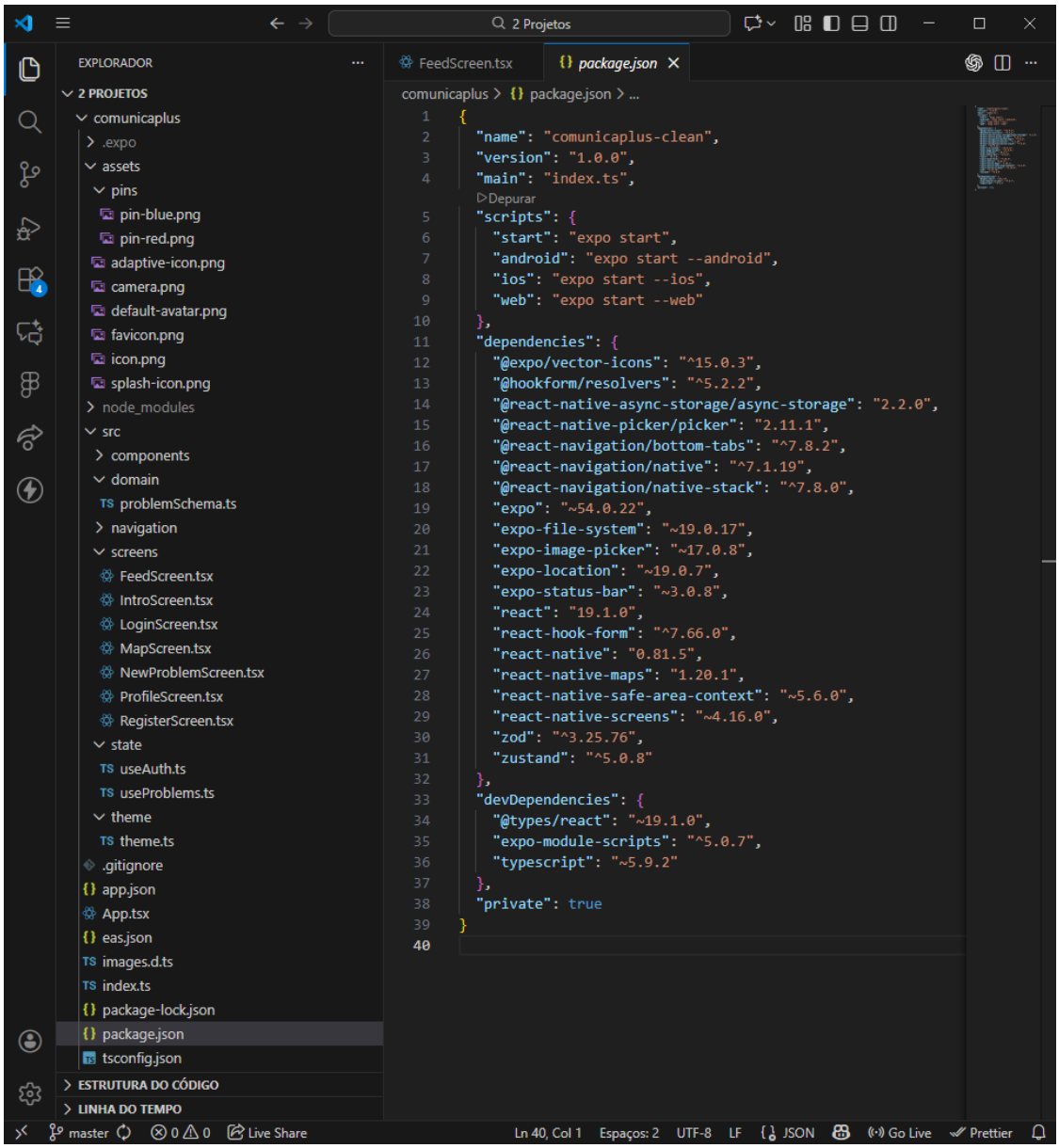
Figura 3 - Estrutura do Projeto Comunica+.



Fonte: autoria própria.

A figura 4 apresenta o arquivo *package.json* do projeto *Comunica+*, no qual estão listadas todas as dependências utilizadas no desenvolvimento da aplicação. Entre elas, incluem-se bibliotecas fundamentais como React Native, Expo, React Navigation, Zustand, React Hook Form, Zod e Expo Location, responsáveis por funcionalidades essenciais como navegação, gerenciamento de estado, validação, captura de imagem e acesso à localização. Essa listagem evidencia a base tecnológica do aplicativo e demonstra a configuração completa do ambiente utilizado no processo de desenvolvimento.

Figura 4 - Dependências instaladas (package.json).



Fonte: autoria própria.

A figura 5 apresenta o arquivo useProblems.ts, responsável pela gestão do estado global da aplicação *Comunica+* utilizando a biblioteca Zustand. Esse módulo centraliza todas as operações relacionadas aos problemas registrados no aplicativo, incluindo criação, listagem, atualização de votos, alteração de status e armazenamento persistente. O registro e a recuperação dos dados ocorrem por meio do *AsyncStorage*, garantindo que as informações permaneçam salvas mesmo após o fechamento do aplicativo. O print evidencia a arquitetura reativa adotada, assegurando organização, consistência e facilidade de manutenção no gerenciamento dos dados.

Figura 5 - Estado Global (useProblems.ts).

```
useProblems.ts
comunicaplus > src > state > useProblems.ts > useProblems > create() callback > vote > arr > problems.map() callbac
import { create } from "zustand";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { ProblemInput } from "../domain/problemSchema";

export type Problem = ProblemInput & {
  id: string;
  status: "Aberto" | "Em andamento" | "Resolvido";
  votes: number;
  votedBy: string[];
  createdAt: number;
};

type Store = {
  problems: Problem[];
  loaded: boolean;
  addProblem: (p: ProblemInput) => Promise<void>;
  vote: (id: string, userId?: string | null) => Promise<boolean>;
  setStatus: (id: string, status: Problem["status"]) => Promise<void>;
  persist: () => Promise<void>;
  load: () => Promise<void>;
  clearAll: () => Promise<void>;
};

const KEY = "@comunicaplus/problems";

function isValidProblem(raw: any): raw is Problem {
  if (!raw) return false;

  const lat = raw.latitude;
  const lng = raw.longitude;

  const latOk = typeof lat === "number" && !Number.isNaN(lat);
  const lngOk = typeof lng === "number" && !Number.isNaN(lng);

  return latOk && lngOk;
}

export const useProblems = create<Store>((set, get) => ({
  problems: [],
  loaded: false,

  load: async () => {
    try {
      const raw = await AsyncStorage.getItem(KEY);
      const parsed = raw ? JSON.parse(raw) : [];

      const arr: Problem[] = Array.isArray(parsed)
        ? parsed.filter(isValidProblem)
        : [];

      set({ problems: arr, loaded: true });
    } catch {
      set({ problems: [], loaded: true });
    }
  },

  persist: async () => {
    await AsyncStorage.setItem(KEY, JSON.stringify(get().problems));
  },

  addProblem: async (p: ProblemInput) => {

    const newItem: Problem = {
      ...p,
      id: Math.random().toString(36).slice(2),
      status: "Aberto",
      votes: 0,
      votedBy: [],
      createdAt: Date.now(),
    };


    if (!isValidProblem(newItem)) {
      console.warn("Tentativa de salvar problema inválido (sem lat/long).");
      return;
    }

    const arr = [newItem, ...get().problems];
    set({ problems: arr });
    await AsyncStorage.setItem(KEY, JSON.stringify(arr));
  },

  vote: async (id: string, userId: string | null | undefined) => {


    if (!userId) return false;

    const arr = get().problems.map((i) => {
      if (i.id !== id) return i;


      if (i.votedBy && i.votedBy.includes(userId)) {
        return i;
      }

      return {
        ...i,
        votes: i.votes + 1,
        votedBy: [...(i.votedBy || []), userId],
      };
    });


    const updated = arr.find(p => p.id === id);
    const original = get().problems.find(p => p.id === id);

    const applied = original && updated && updated.votes > original.votes;

    set({ problems: arr });
    await AsyncStorage.setItem(KEY, JSON.stringify(arr));

    return Boolean(applied);
  },

  setStatus: async (id: string, status: Problem["status"]) => {
    const arr = get().problems.map((i) =>
      i.id === id ? { ...i, status } : i
    );
    set({ problems: arr });
    await AsyncStorage.setItem(KEY, JSON.stringify(arr));
  },

  clearAll: async () => {
    set({ problems: [] });
    await AsyncStorage.removeItem(KEY);
  },
}));
```

Fonte: autoria própria.

A figura 6 apresenta o arquivo problemSchema.ts, responsável pela definição das regras de validação utilizadas no formulário do *Comunica+*. Esse schema, construído com a biblioteca Zod, especifica os requisitos mínimos para cada campo — como tamanho, formato e obrigatoriedade — garantindo a integridade dos dados inseridos pelo usuário. O print demonstra como a aplicação assegura consistência e confiabilidade já na etapa de entrada das informações.

Figura 6 - Schema de validação (problemSchema.ts).

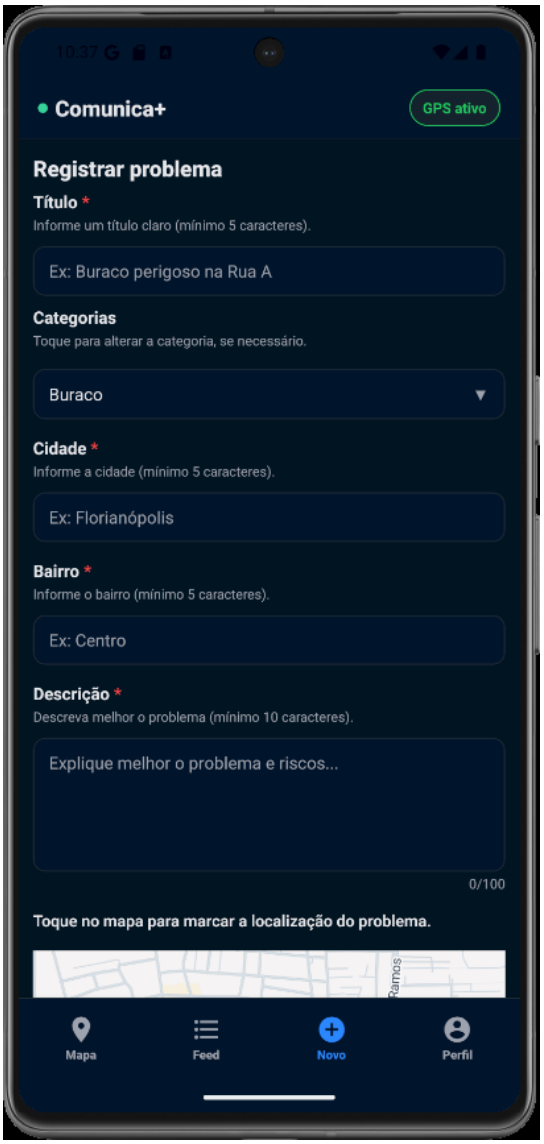
```
TS problemSchema.ts ×
comunicaplus > src > domain > TS problemSchema.ts > ...
1   // domain/problemSchema.ts
2   import { z } from "zod";
3
4   // evita caracteres muito bizarros (emojis aleatórios, símbolos estranhos, etc.)
5   const weirdCharsRegex = /[^\wÀ-Öø-öø-ÿ0-9\s.,\-!?]/;
6
7   export const problemSchema = z.object({
8     title: z
9       .string()
10      .trim()
11      .min(5, "Informe um título com pelo menos 5 caracteres.")
12      .max(50, "O título pode ter no máximo 50 caracteres.")
13      .refine((v) => !weirdCharsRegex.test(v), {
14        message: "Evite caracteres especiais estranhos no título.",
15      }),
16
17    category: z
18      .string()
19      .trim()
20      .min(1, "Selecione uma categoria."),
21
22    city: z
23      .string()
24      .trim()
25      .min(5, "Informe a cidade (mínimo 5 caracteres).")
26      .max(50, "A cidade pode ter no máximo 50 caracteres.")
27      .refine((v) => !weirdCharsRegex.test(v), {
28        message: "Evite caracteres especiais estranhos na cidade.",
29      }),
30
31    // bairro agora é obrigatório
32    neighborhood: z
33      .string()
34      .trim()
35      .min(5, "Informe o bairro (mínimo 5 caracteres).")
36      .max(50, "O bairro pode ter no máximo 50 caracteres.")
37      .refine((v) => !weirdCharsRegex.test(v), {
38        message: "Evite caracteres especiais estranhos no bairro.",
39      }),
40
41    description: z
42      .string()
43      .trim()
44      .min(10, "Descreva melhor o problema (mínimo 10 caracteres).")
45      .max(100, "A descrição pode ter no máximo 100 caracteres.")
46      .refine((v) => !weirdCharsRegex.test(v), {
47        message: "Evite caracteres especiais estranhos na descrição.",
48      }),
49
50    latitude: z.number({
51      required_error:
52        "Selecione um ponto no mapa ou use o botão 'Usar minha localização'.",
53    }),
54
55    longitude: z.number({
56      required_error:
57        "Selecione um ponto no mapa ou use o botão 'Usar minha localização'.",
58    }),
Ln 64, Col 1   Espaços: 2   UTF-8   CRLF   TypeScript   Go Live   Prettier
```

Fonte: autoria própria.

A figura 7 e 8 apresenta a tela de registro de novos problemas do aplicativo *Comunica+*, desenvolvida no componente NewProblemScreen. Esse formulário reúne todos os campos necessários para que o usuário cadastre uma ocorrência, incluindo título, categoria, cidade, bairro, descrição, localização no mapa e imagem opcional. O layout foi projetado para ser intuitivo e responsivo, incorporando validações em tempo real pelo *React Hook Form* e *Zod*, além da integração com câmera, galeria e mapa interativo. O print evidencia a estrutura

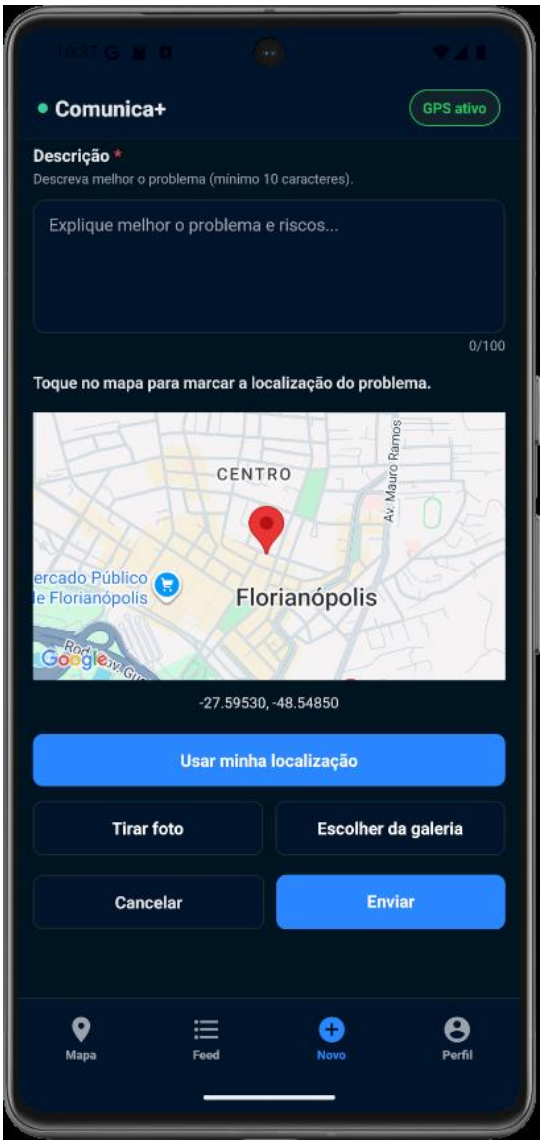
completa do processo de entrada de dados, etapa fundamental para o funcionamento da aplicação.

Figura 7 - Tela do formulário vazia.



Fonte: autoria própria.

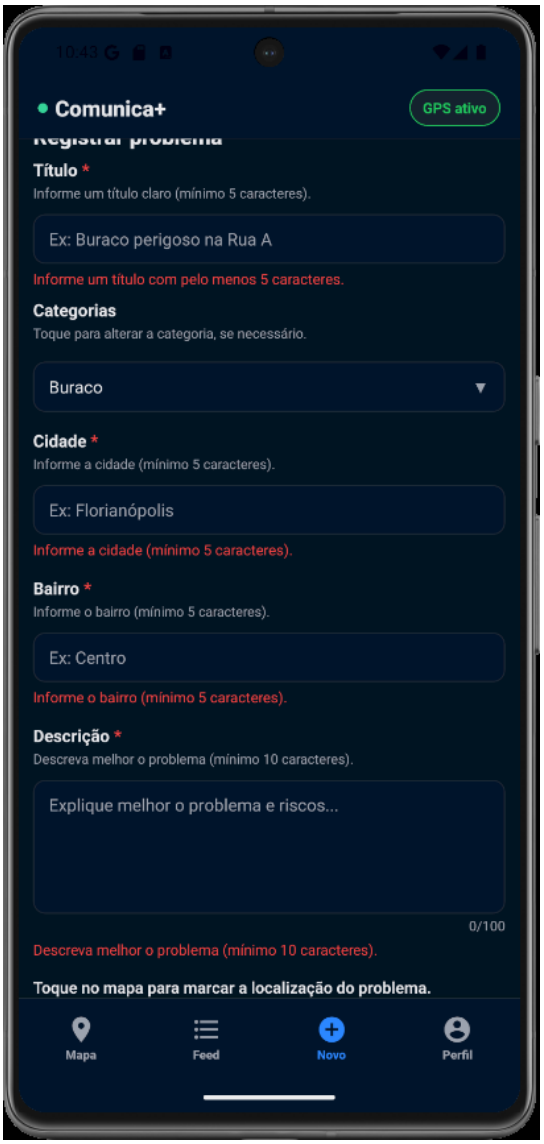
Figura 8 - Tela do formulário vazia.



Fonte: autoria própria.

A figura 9 apresenta as mensagens de validação exibidas aos usuários durante o preenchimento do formulário de registro do *Comunica+*. Essas mensagens são geradas automaticamente pelo esquema de validação definido com React Hook Form e Zod, garantindo que todos os campos obrigatórios atendam aos critérios mínimos antes do envio. O print demonstra o funcionamento das regras de consistência, assegurando a qualidade dos dados cadastrados e prevenindo erros de entrada por parte do usuário.

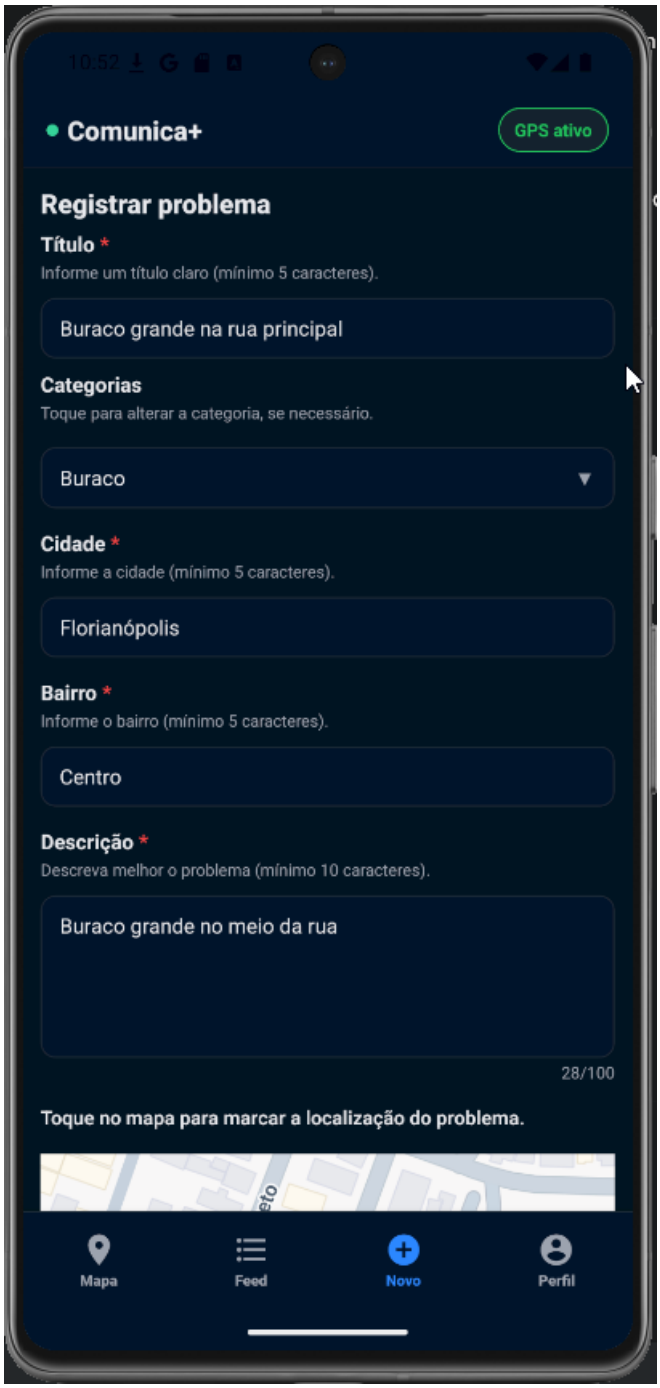
Figura 9 - Mensagens de erro de validação.



Fonte: autoria própria.

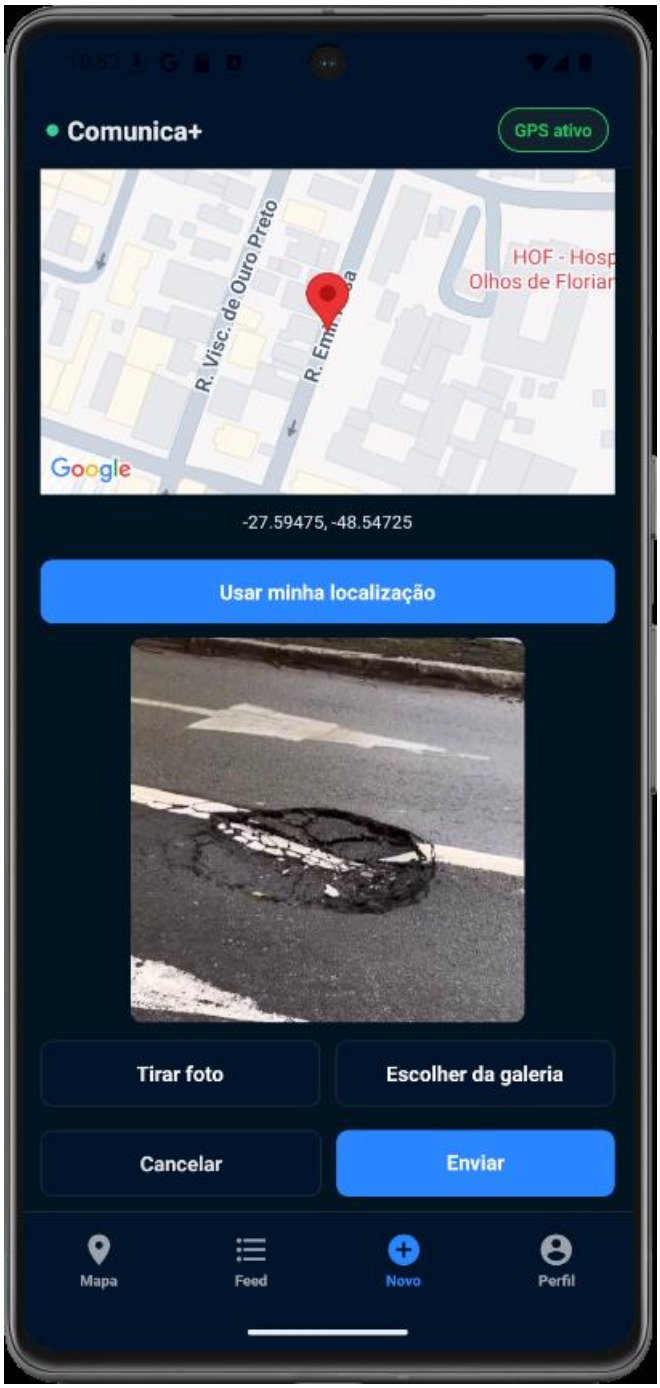
A figura 9 e 10 apresenta o formulário de registro do *Comunica+* devidamente preenchido com todas as informações solicitadas. O print evidencia o correto funcionamento das validações, bem como a organização dos campos, seleção de categoria, localização marcada no mapa e imagem anexada. Esse registro completo demonstra que o usuário pode enviar um problema sem erros, confirmando a eficácia das regras de preenchimento e a usabilidade da interface desenvolvida.

Figura 10 - Formulário preenchido corretamente.



Fonte: autoria própria.

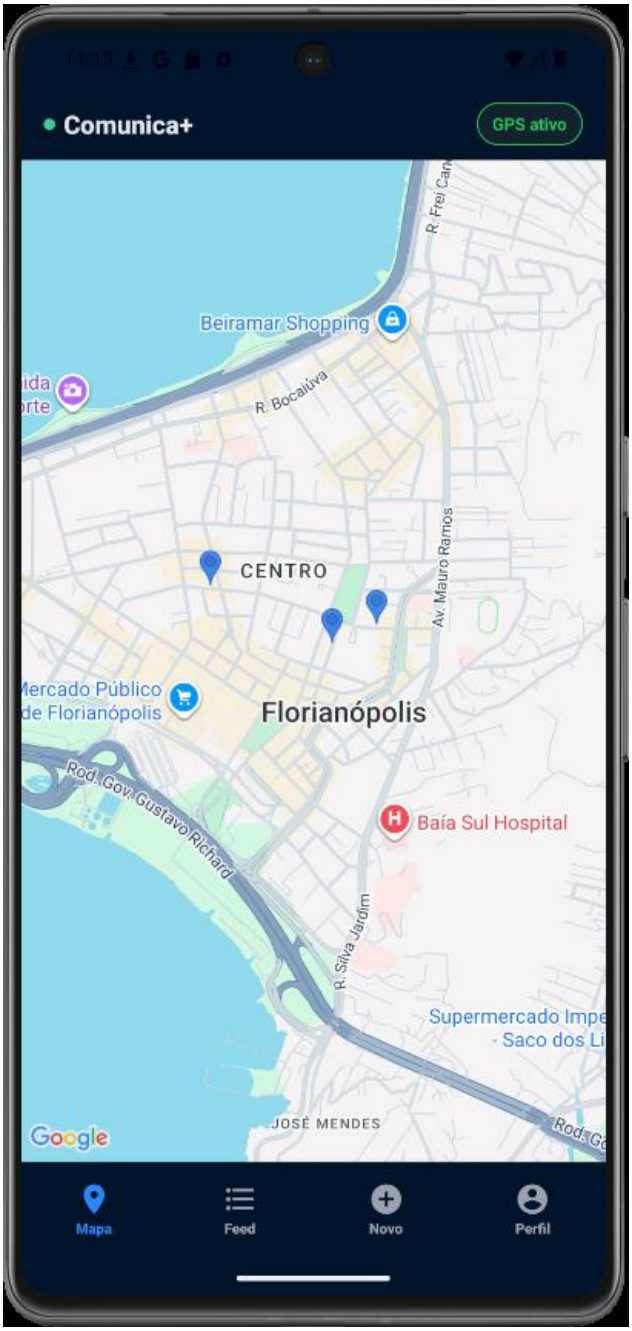
Figura 11 - Formulário preenchido corretamente.



Fonte: autoria própria.

A figura 12 apresenta o mapa interativo do *Comunica+* exibindo múltiplos pins, cada um representando um problema registrado pelos usuários. Cada marcador utiliza uma imagem personalizada e contém informações essenciais, como título, categoria e localização. O print evidencia o funcionamento da visualização geográfica das ocorrências, permitindo que o usuário identifique facilmente os pontos de interesse e navegue até cada registro.

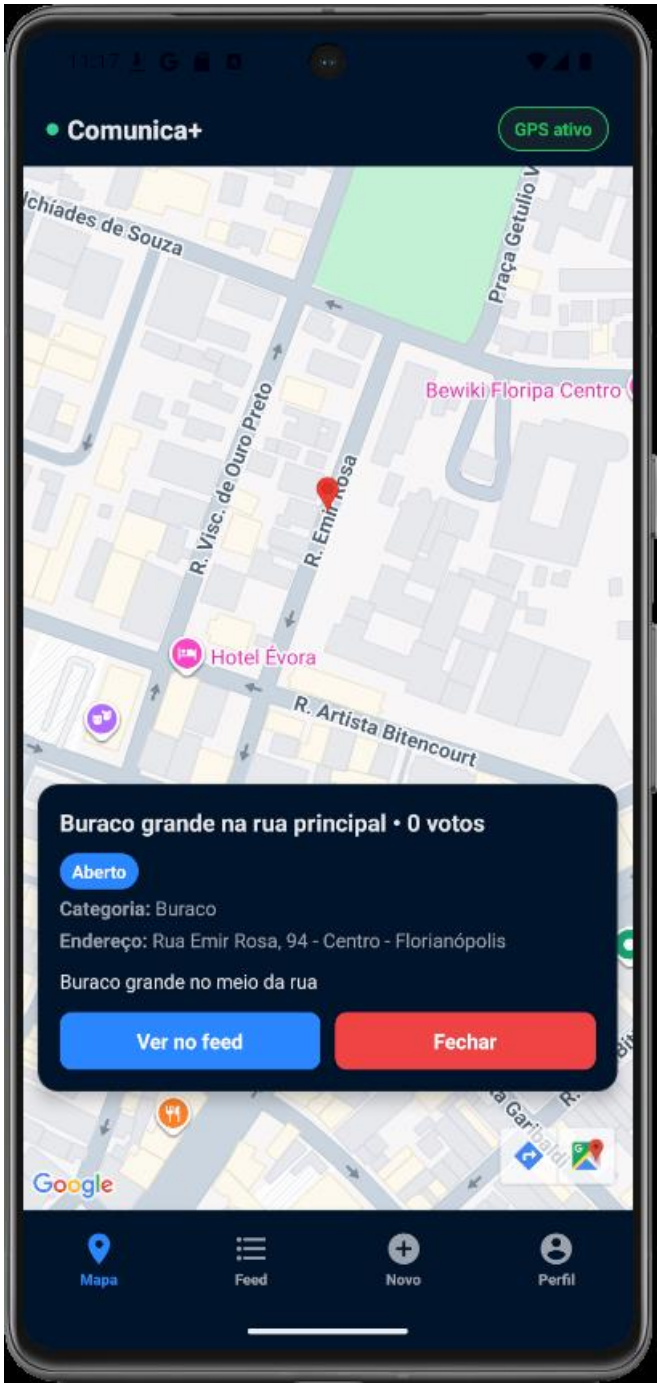
Figura 12 - Mapa com múltiplos pins.



Fonte: autoria própria.

A figura 13 apresenta o card exibido ao selecionar um pin no mapa do *Comunica+*, mostrando um resumo das informações do problema sem expansão completa. Nesse estado, o usuário visualiza título, categoria, endereço e uma prévia da descrição, mantendo a interface limpa e objetiva. O print demonstra a funcionalidade de visualização rápida, permitindo que o usuário consulte detalhes essenciais.

Figura 13 - Card do pin aberto.



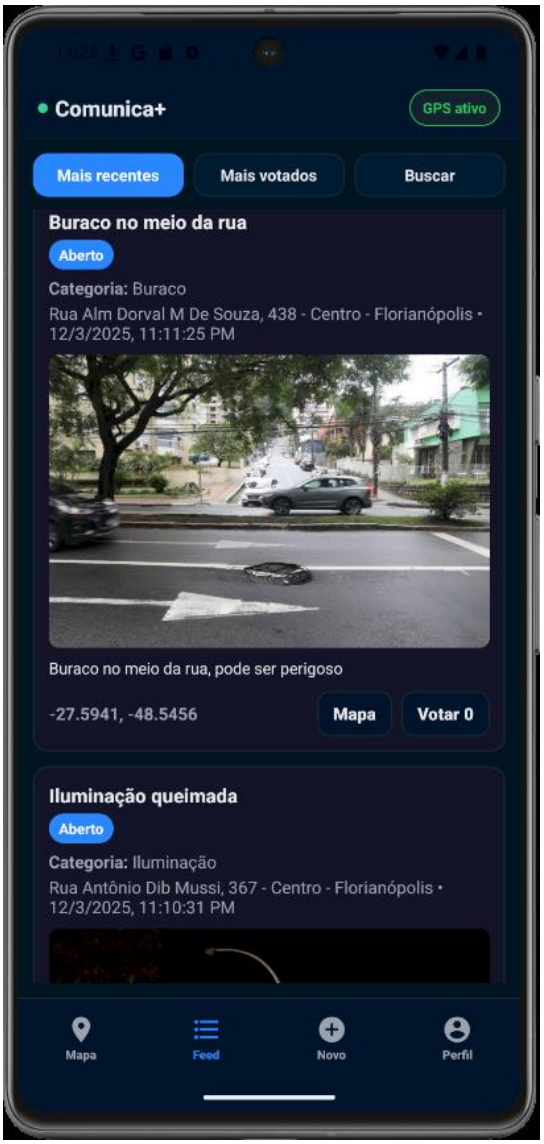
Fonte: autoria própria.

A figura 14 e 15 apresenta a tela de feed do *Comunica+*, onde diversos problemas cadastrados são exibidos em forma de lista. Cada card contém informações essenciais, como título, categoria, endereço, status, imagem (quando disponível) e quantidade de votos. O print evidencia a organização visual da listagem, permitindo que o usuário navegue pelas ocorrências de maneira clara, rápida e eficiente.

Figura 14 - Feed com vários problemas listados.

Fonte: autoria própria.

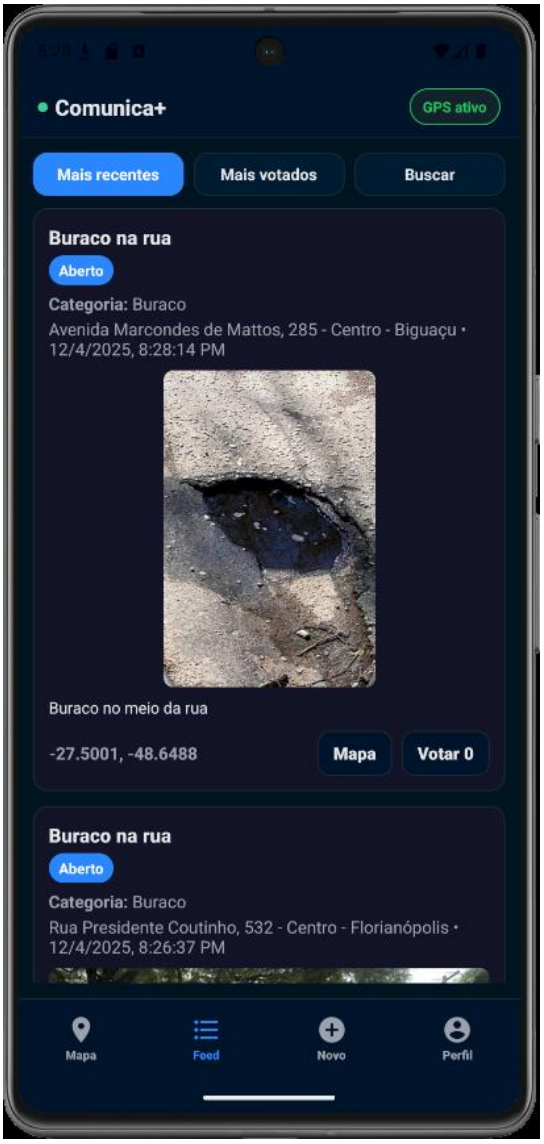
Figura 15 - Feed com vários problemas listados.



Fonte: autoria própria.

A figura 16 apresenta um card do feed contendo uma imagem registrada em orientação vertical, demonstrando que o aplicativo adapta automaticamente o aspecto da mídia enviada pelo usuário. O ajuste preserva a proporção original da imagem, evitando distorções e garantindo boa visualização dentro do layout do card. Esse print evidencia a preocupação com responsividade e usabilidade na exibição de diferentes formatos de imagem no Comunica+.

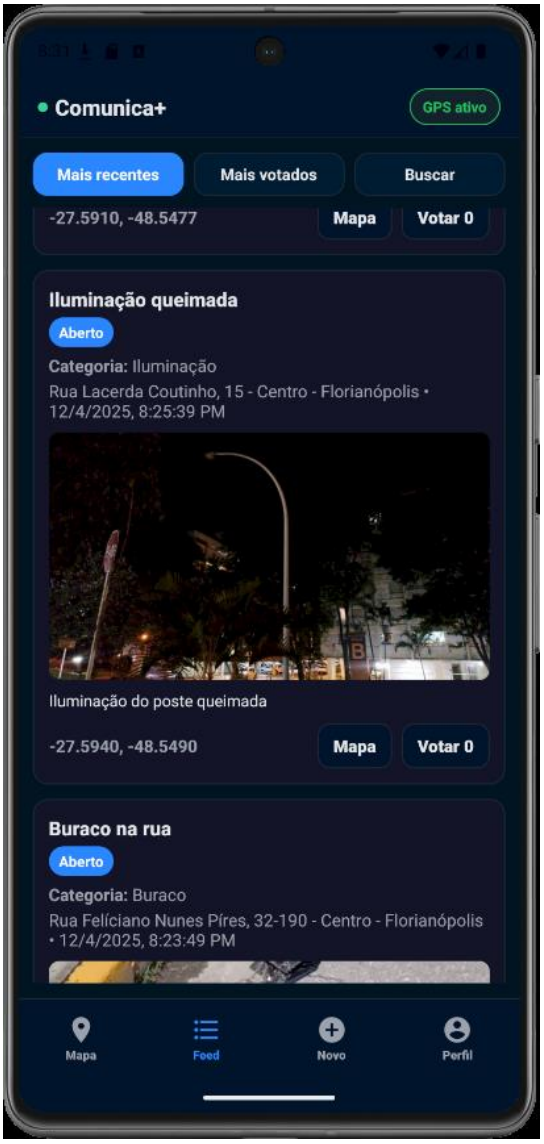
Figura 16- Card com imagem vertical.



Fonte: autoria própria.

A figura 17 apresenta um card contendo uma imagem em orientação horizontal, demonstrando a capacidade do aplicativo de ajustar automaticamente a largura da mídia para ocupar todo o espaço disponível no card, mantendo a proporção correta. Esse print evidencia a flexibilidade do *Comunica+* em lidar com diferentes formatos de imagem, garantindo boa apresentação visual e experiência consistente ao usuário.

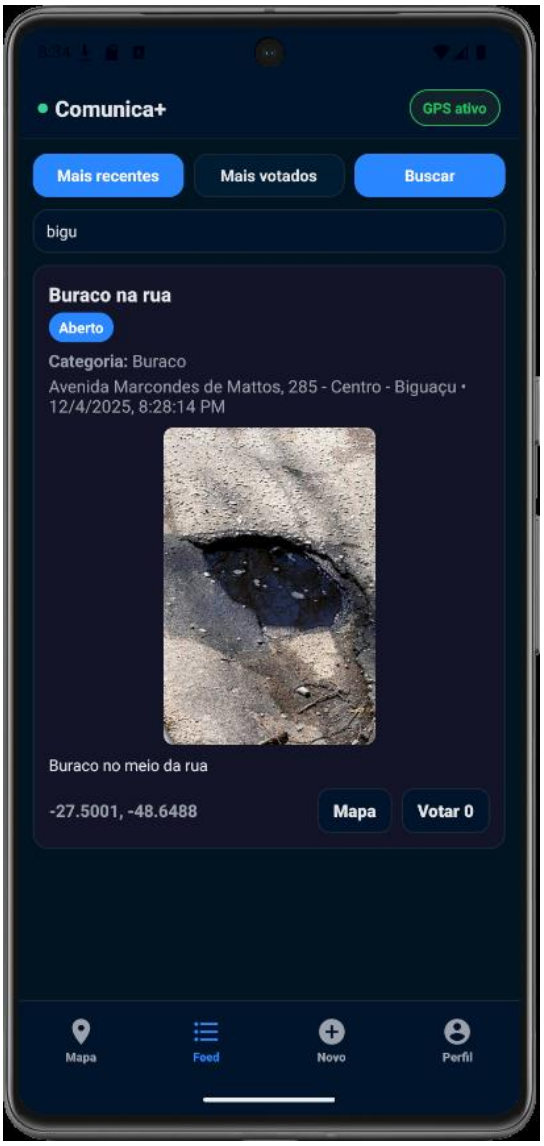
Figura 18 - Card com imagem horizontal.



Fonte: autoria própria.

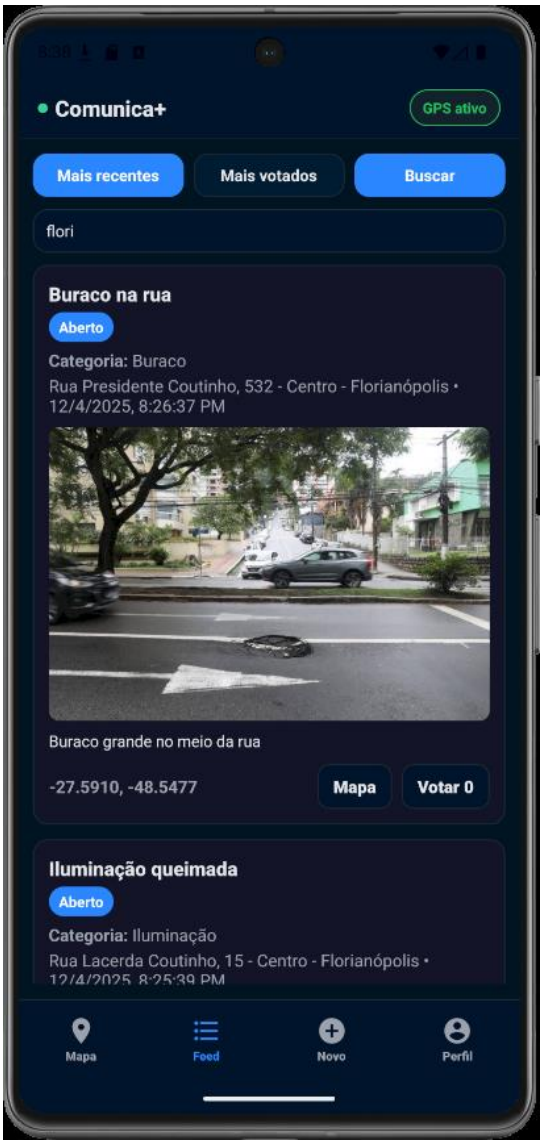
A figura 18, 19 e 20 apresenta a funcionalidade de busca ativa no feed do *Comunica+*, filtrando os problemas cadastrados de acordo com o termo digitado pelo usuário. O resultado exibido demonstra que o sistema aplica corretamente o filtro por título, descrição, endereço, bairro ou cidade, retornando apenas os itens correspondentes. Esse print comprova o pleno funcionamento da busca dinâmica e sua importância para facilitar a localização rápida de ocorrências específicas.

Figura 18 - Busca funcionando.



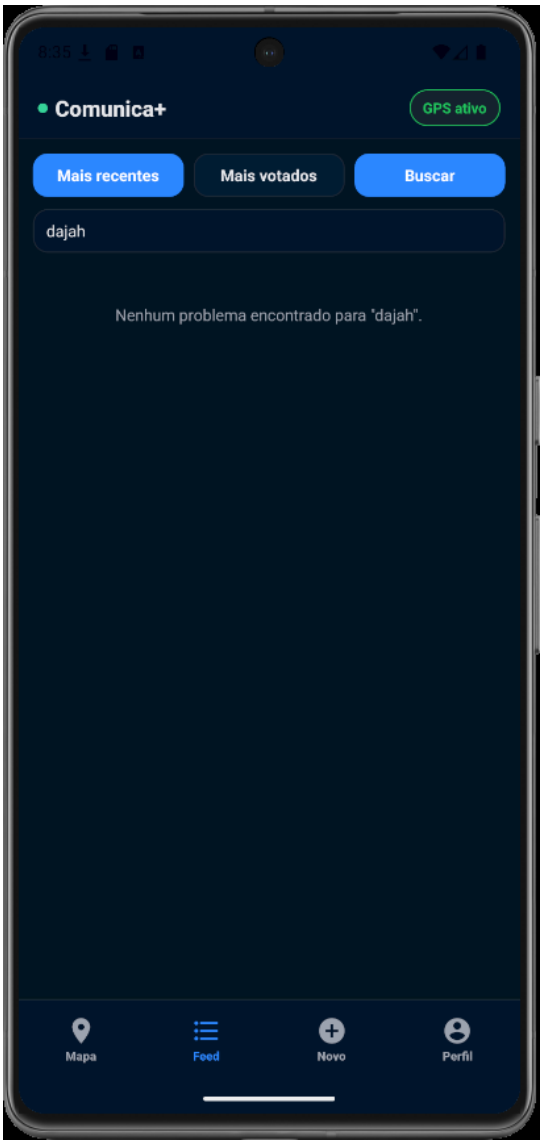
Fonte: autoria própria.

Figura 19 - Busca funcionando.



Fonte: autoria própria.

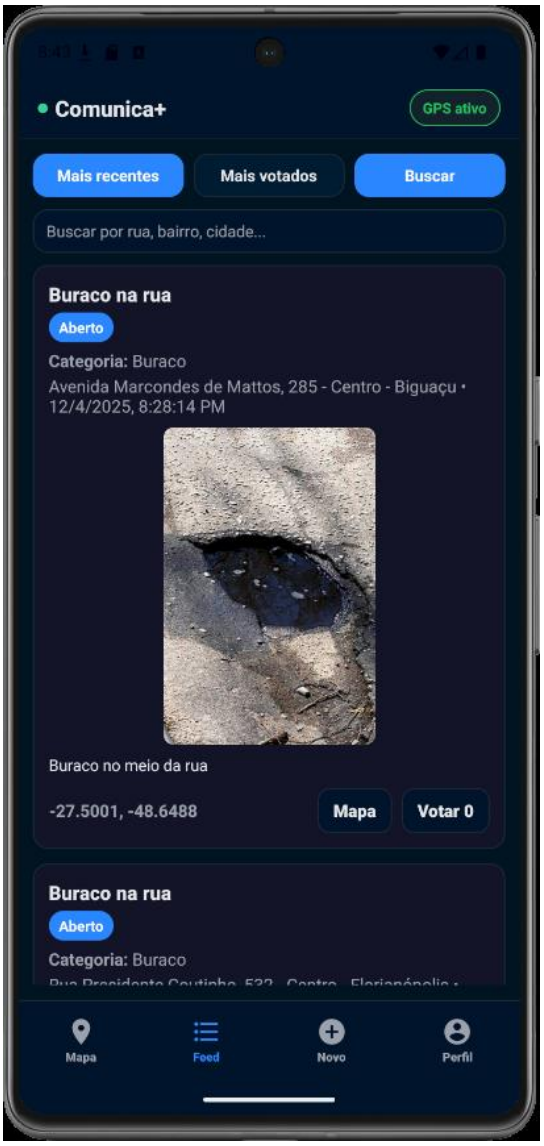
Figura 20 - Busca funcionando.



Fonte: autoria própria.

A figura 21 apresenta a funcionalidade de ordenação do feed pela opção “Mais votados”, reorganizando os cards de acordo com a quantidade de votos recebida por cada problema. O print evidencia que o aplicativo prioriza no topo da lista as ocorrências consideradas mais relevantes pelos usuários, facilitando a visualização de demandas urgentes da comunidade. Essa funcionalidade reforça o caráter participativo do *Comunica+*, permitindo que os cidadãos destaquem problemas que necessitam de atenção imediata.

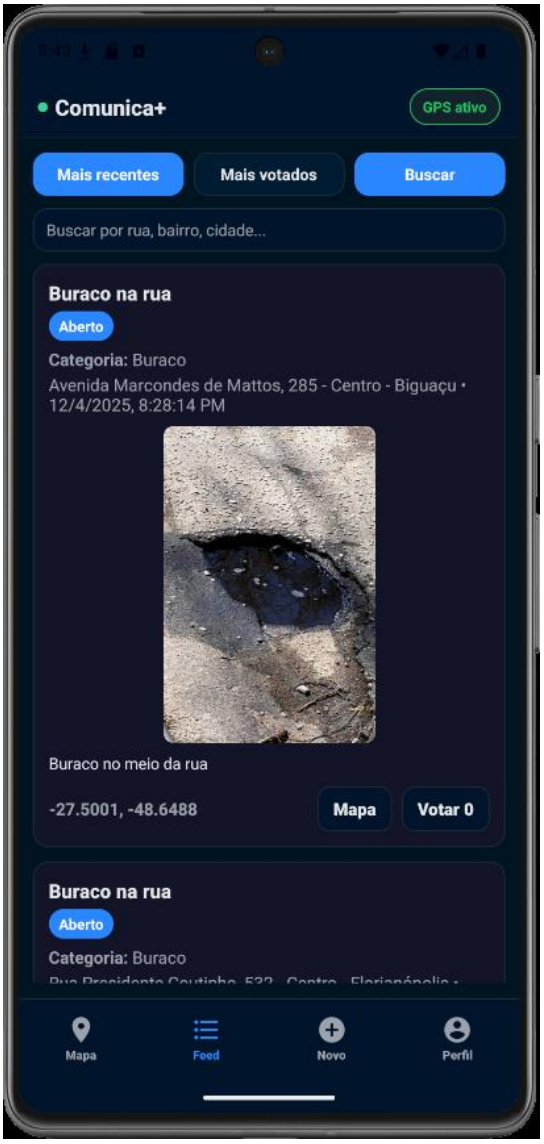
Figura 21 - Ordenação “Mais votados”.



Fonte: autoria própria.

A figura 22 apresenta o botão de voto antes da interação do usuário, exibido no estado padrão, sem destaque visual. Esse print demonstra como o *Comunica+* apresenta a opção de votação de forma clara e acessível, permitindo que o usuário identifique facilmente a ação disponível. O estado inicial do botão é fundamental para evidenciar o comportamento esperado antes da contagem de votos ser atualizada.

Figura 22 - Botão de voto antes de clicar.



Fonte: autoria própria.

A figura 23 apresenta o estado do card após o usuário realizar um voto, exibindo a contagem de votos atualizada imediatamente na interface. O print evidencia o correto funcionamento da lógica de votação do *Comunica+*, demonstrando que o sistema registra a interação do usuário, impede votos duplicados e atualiza a interface de forma dinâmica. Essa funcionalidade reforça o caráter participativo do aplicativo e valida a integração entre interface, estado global e regras de negócio.

Figura 23 - Depois de votar.

Fonte: autoria própria.

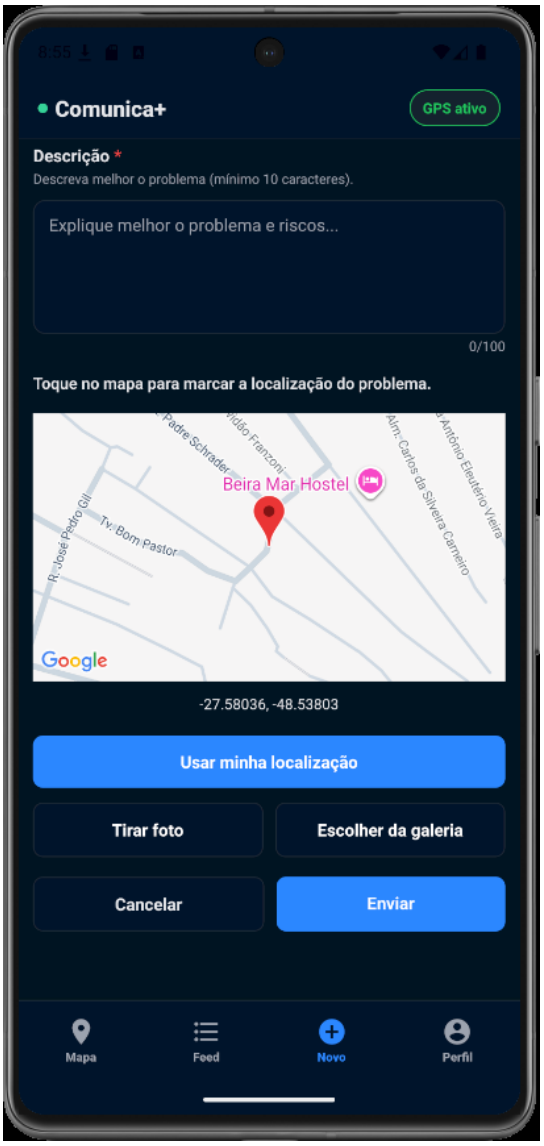
A figura 24 apresenta a mensagem exibida pelo *Comunica+* quando o usuário tenta votar novamente em um problema já votado anteriormente. O print demonstra que o aplicativo possui um mecanismo de proteção contra votos duplicados, garantindo a integridade da contagem e impedindo manipulações indevidas. Essa validação é essencial para assegurar a confiabilidade do sistema de priorização de problemas, mantendo a participação justa e transparente dentro da plataforma.

Figura 24 - Tentativa de voto duplicado.

Fonte: autoria própria.

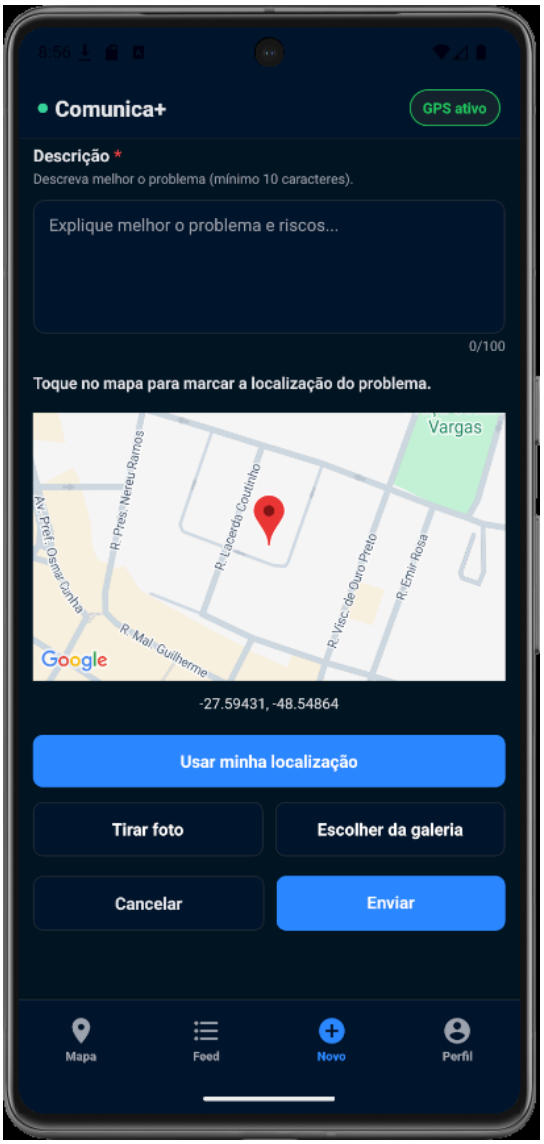
A figura 25 e 26 apresenta o formulário de registro de problemas já preenchido com as coordenadas geográficas obtidas a partir do mapa ou da função “Usar minha localização”. Esse print demonstra o correto funcionamento da integração entre o componente de mapa e o formulário do *Comunica+*, garantindo que cada ocorrência registrada possua uma localização precisa. A presença das coordenadas reforça a confiabilidade do sistema e a rastreabilidade das informações inseridas pelo usuário.

Figura 25 - Antres de “Usar minha localização”.



Fonte: autoria própria.

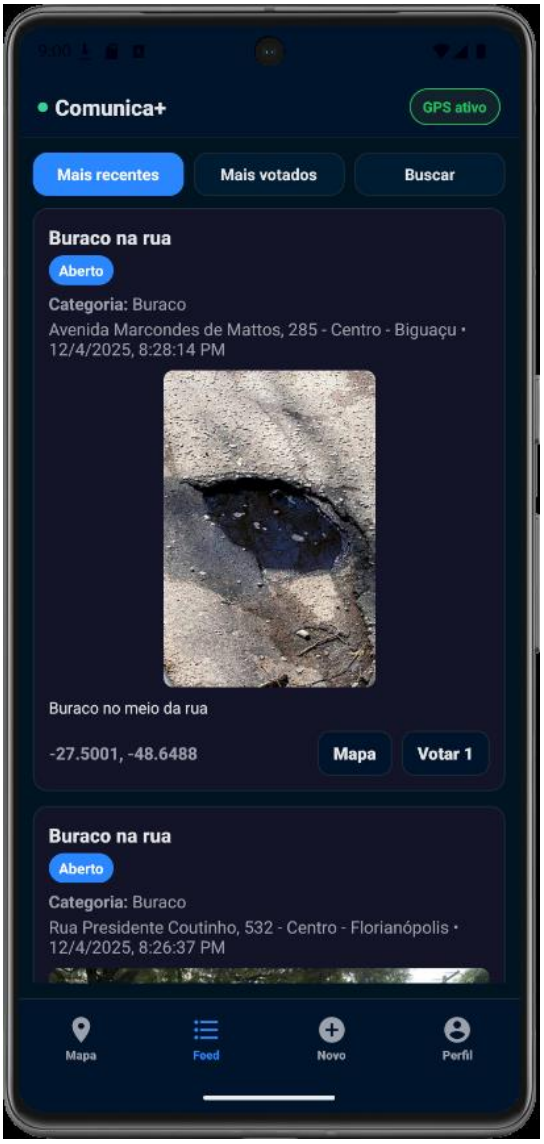
Figura 26 - Depois de “Usar minha localização”.



Fonte: autoria própria.

A figura 27 apresenta um card no feed exibindo o endereço completo do problema, resultado do processo de geocodificação reversa implementado no aplicativo. O print demonstra que o *Comunica+* transforma automaticamente as coordenadas registradas em um endereço legível, facilitando a compreensão do local da ocorrência pelos usuários. Essa funcionalidade melhora a usabilidade e reforça a precisão das informações apresentadas no feed.

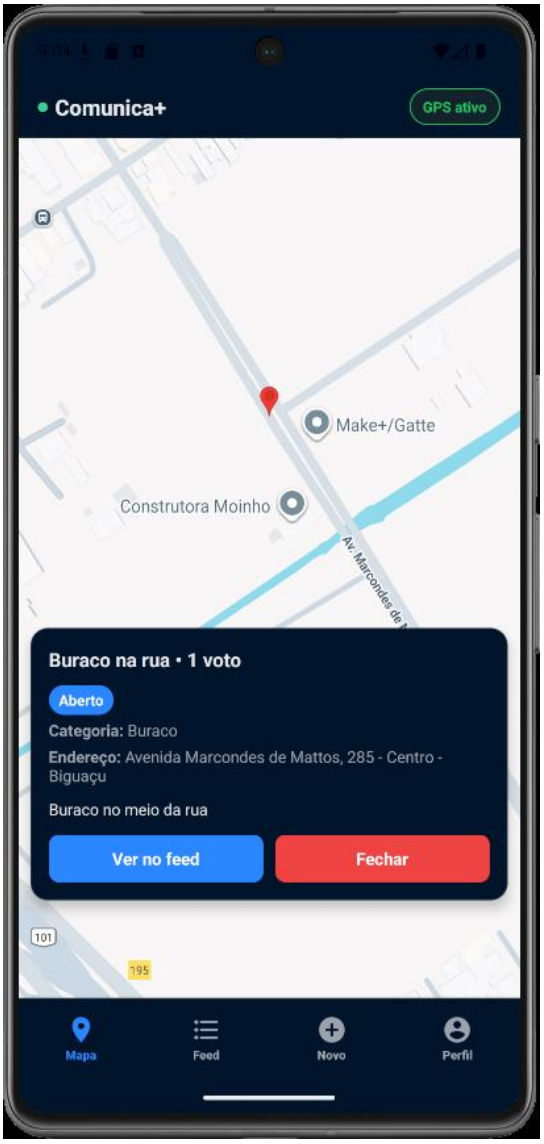
Figura 27 - Endereço no feed.



Fonte: autoria própria.

A figura 28 apresenta o endereço exibido no card associado ao pin selecionado no mapa, resultado da conversão automática das coordenadas geográficas em um endereço legível. Esse print demonstra que o *Comunica+* utiliza geocodificação reversa para fornecer ao usuário informações claras sobre a localização exata do problema, aprimorando a experiência de navegação e a precisão dos dados apresentados na interface de mapa.

Figura 28 - Endereço no mapa



Fonte: autoria própria.

6 SCRUM Configurado

A metodologia Scrum foi adotada no desenvolvimento do projeto *Comunica+* por sua capacidade de organizar o trabalho em ciclos curtos e entregas contínuas, favorecendo o planejamento incremental e o controle de qualidade. Essa metodologia ágil foi configurada e gerenciada através da ferramenta Trello, que serviu como quadro de acompanhamento das atividades, tarefas e prazos de cada sprint.

O Daily Scrum do primeiro sprint do projeto *Comunica+* teve como foco principal o planejamento, organização inicial e configuração das bases do aplicativo. As reuniões diárias foram utilizadas para acompanhar o progresso, identificar obstáculos e garantir a conclusão das metas previstas no cronograma.

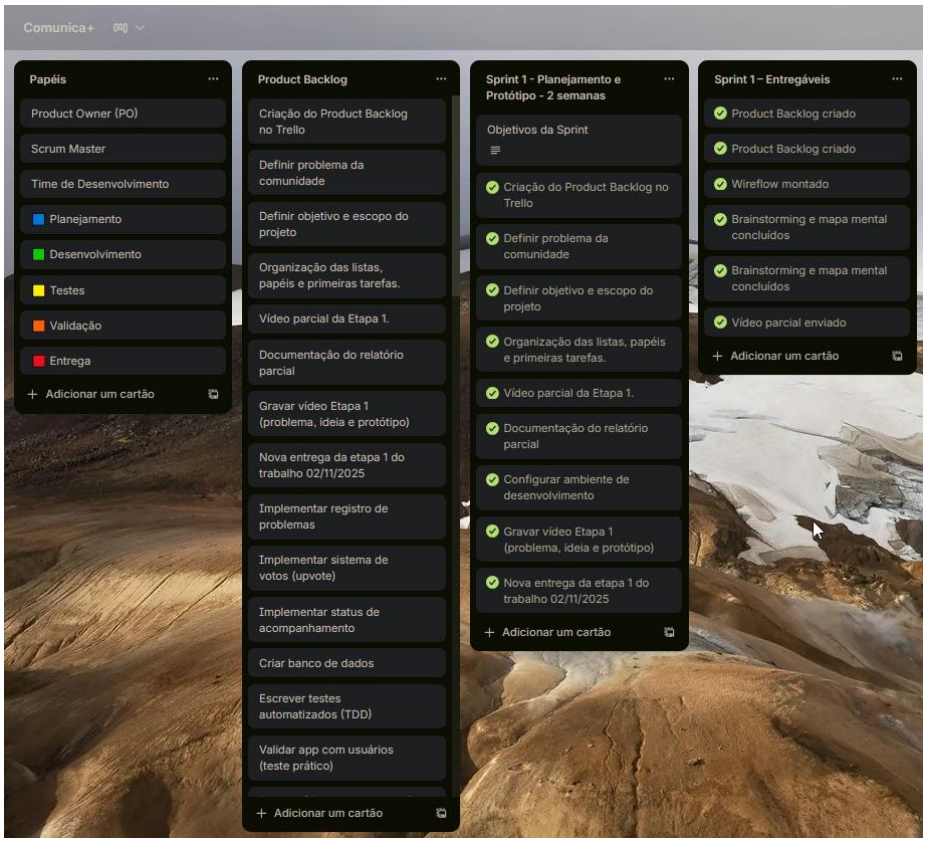
Período do Sprint:

De **21/10/2025** a **02/11/2025**

Objetivo do Sprint:

Definir o escopo e os objetivos do projeto, organizar o backlog no Trello, preparar o ambiente de desenvolvimento e entregar o vídeo parcial da Etapa 1.

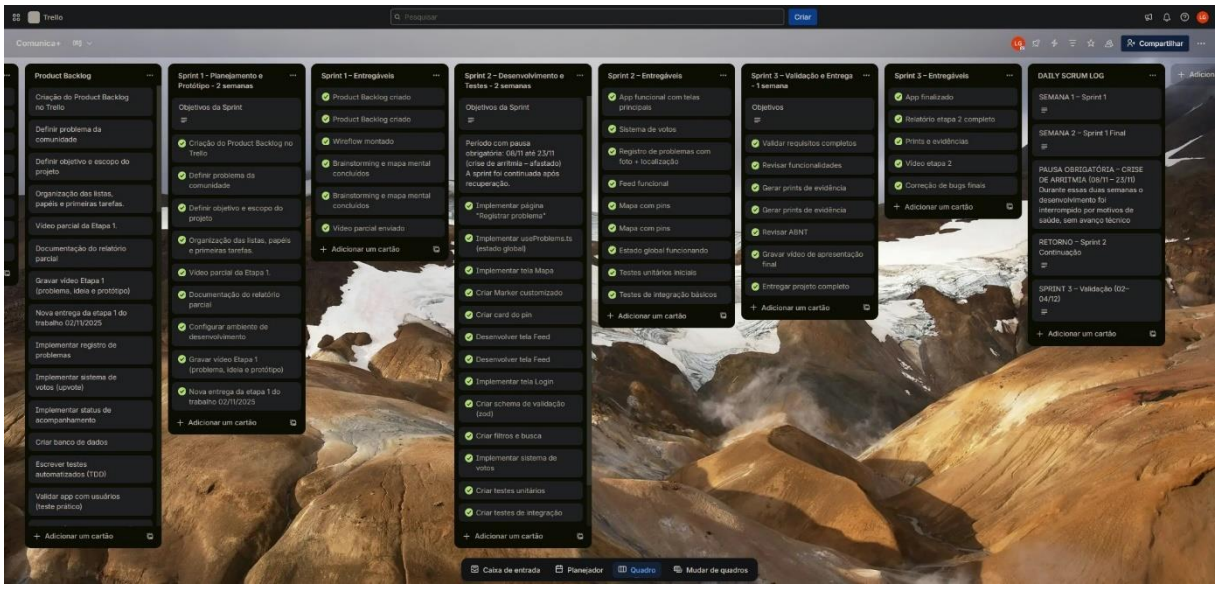
Figura 29- Trello Scrum



Fonte: Feito pelo autor.

A Parte 2 do projeto manteve a metodologia SCRUM aplicada na etapa anterior, evoluindo o desenvolvimento para fases práticas, incluindo criação do aplicativo, implementação das funcionalidades, testes e validação final. A seguir, estão descritos os artefatos Scrum utilizados durante toda a segunda etapa do projeto.

Figura 9 - Scrum Trello



Fonte: autoria própria.

Link Trello: https://trello.com/invite/b/68f40013c08745acfb01e2b9/ATTI0aaaf78b61fc92807ea7b99a0f7f906e59F4F98E/comunica

7 Protótipos

Link do Figma: https://www.figma.com/design/jLbtxeTaIycErGQp5gx0pZ/Comunica-?node-id=0-1&t=xKhVkBIUBOEubVo6-1

8 Repositório GitHub

Link do repositório: https://github.com/lzinho-g/comunicaplus.git

9 Apk do aplicativo

Link do apk: https://expo.dev/accounts/luizhyg/projects/comunicaplus/builds/c119d181-ad42-40c9-9b51-4a42a2822dc6

10 Vídeo de Apresentação

10.1 Etapa 1

Link vídeo: https://youtu.be/BJkqR-MNkPs

10.2 Etapa 2

Link vídeo: https://youtu.be/8E4h6LS_js0

11 Conclusão

O desenvolvimento do aplicativo *Comunica+* permitiu integrar práticas modernas de engenharia de software, metodologias ágeis e tecnologias mobile para a construção de uma solução funcional voltada ao engajamento cidadão na identificação de problemas urbanos. Ao longo do projeto, foi possível aplicar de maneira consistente o framework Scrum, estruturando o trabalho em Sprints, definindo metas claras e registrando impedimentos, o que garantiu controle do progresso e capacidade de adaptação às demandas surgidas durante o processo, incluindo imprevistos reais enfrentados no período.

A implementação baseada em princípios de TDD e validação contínua assegurou maior qualidade ao software, permitindo que cada funcionalidade — como registro de problemas, geolocalização, mapa interativo, feed de ocorrências e sistema de votos — fosse testada, ajustada e refinada antes da integração final. O uso de tecnologias como React Native, Expo,

TypeScript, Zustand e Zod contribuiu para a criação de uma arquitetura sólida, modular e escalável, facilitando a manutenção e a evolução futura do projeto.

Os testes funcionais, de interface e de fluxo completo demonstraram que o *Comunica+* cumpre adequadamente seus requisitos, apresentando desempenho estável e experiência de uso intuitiva. As evidências coletadas, por meio de prints e registros de execução, reforçam a confiabilidade do produto entregue.

Dessa forma, conclui-se que o *Comunica+* alcançou seu objetivo central: oferecer uma ferramenta simples, eficiente e acessível para promover a comunicação entre cidadãos e gestão pública. O projeto também proporcionou significativo aprendizado prático sobre desenvolvimento mobile, testes, validação e organização ágil, consolidando competências essenciais para a atuação profissional na área de tecnologia.

12 Referências Bibliográficas

ALVES, W. C.; LIMA, P. H. M.; CARVALHO, D. A. **Desenvolvimento de aplicações móveis multiplataforma com React Native**. Revista de Engenharia e Tecnologia, v. 11, n. 1, p. 45–57, 2023.

BECK, Kent. *Test-Driven Development: By Example.* Boston: Addison-Wesley Professional, 2003.

BECK, K. **Test-Driven Development: Desenvolvimento Guiado por Testes**. 1. ed. São Paulo: Novatec Editora, 2014.

EXPO. *Expo Documentation.* Disponível em: https://docs.expo.dev. Acesso em: 1 dez. 2025.

FREITAS, M. A.; SILVA, G. J. **Aplicações móveis e o uso de frameworks híbridos: uma análise prática do React Native**. Revista Eletrônica de Sistemas de Informação e Gestão Tecnológica, v. 9, n. 2, p. 25–38, 2022.

KOSCIANSKI, A.; SOARES, M. S. **Qualidade de Software: Aprenda as metodologias e técnicas mais modernas**. 3. ed. São Paulo: Novatec Editora, 2018.

LARMAN, C. **Utilizando UML e Padrões: Uma introdução à análise e ao projeto orientado a objetos e ao desenvolvimento iterativo**. 3. ed. Porto Alegre: Bookman, 2007.

MIRO. *Miro Board – Ferramenta de criação de mapas mentais e brainstorming*. Disponível em:https://miro.com/welcomeonboard/T01FL1pJcjB1cVFra082TDJTNm85ZTJ2aXl2TUhGZXNFNVFyNHJWV0Q1RFYvdFBrdXloOTlqSnJUNkNMOVQ1R2toSUZOV29zSU90enowTDZKbHdCQkoxbXc3MEZaQzlGcUd6SjViUkdMaHNadWN6dVpaWHA4ZFh0aitOenNXZ2Z3VHhHVHd5UWtSM1BidUtUYmxycDRnPT0hdjE=?share_link_id=412377492006. Acesso em: **27 out. 2025**.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: Uma abordagem profissional**. 9. ed. Porto Alegre: AMGH Editora, 2021.

REACT HOOK FORM. *React Hook Form Documentation.* Disponível em: https://react-hook-form.com. Acesso em: 1 dez. 2025.

REACT NATIVE. *React Native Documentation.* Disponível em: https://reactnative.dev. Acesso em: 1 dez. 2025.

SCHWABER, K.; SUTHERLAND, J. **Guia do Scrum: O guia definitivo para o Scrum – As regras do jogo**. Tradução oficial do Scrum.org. Versão 2020. Disponível em: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-BR.pdf

SILVA, A. F.; SANTOS, R. L. **Metodologias Ágeis no Desenvolvimento de Software: Um estudo sobre o uso do Scrum em projetos acadêmicos**. Revista Brasileira de Computação Aplicada, v. 12, n. 3, p. 75–90, 2020.

SOMMERVILLE, Ian. *Engenharia de Software.* 10. ed. São Paulo: Pearson Education, 2019.

SOUZA, P. R. **Cidadania Digital: O papel das tecnologias na participação social**. Revista de Inovação e Tecnologia, v. 8, n. 2, p. 102–115, 2021.

TRELLO. **O que é Trello e como usar o método Kanban na prática**. Atlassian Brasil, 2024. Disponível em: https://trello.com/pt-BR. Acesso em: 21 out. 2025.

ZOD. *Zod Documentation.* Disponível em: https://zod.dev. Acesso em: 1 dez. 2025.